

学校代码: 10286

分类号: TP311

密 级: 公开

U D C: 004.4

学 号: 174360



东南大学

工程硕士学位论文

基于可验证随机函数的拜占庭容错共识

算法的改进与实现

(学位论文形式: 应用研究)

研究生姓名: 李希之

导师姓名: 李必信

叶 鹏

申请学位类别 工程硕士 学位授予单位 东南大学

工程领域名称 软件工程 论文答辩日期 2019 年 8 月 28 日

研究方向 区块链 学位授予日期 20 年 月 日

答辩委员会主席 徐立臻 评 阅 人 翟玉庆 徐学永

2019 年 月 日

東南大學

工程硕士学位论文

基于可验证随机函数的拜占庭容错共识算法 的改进与实现

专业名称: 软件工程

研究生姓名: 李希之

导师姓名: 李必信

叶鹏

IMPROVEMENT AND IMPLEMENTATION OF BYZANTINE FAULT TOLERANCE CONSENSUS ALGORITHM BASED ON VERIFIABLE RANDOM FUNCTION

A Thesis Submitted to

Southeast University

For the Professional Degree of Master of Engineering

BY

Li Xi-zhi

Supervised by

Prof. Li Bi-xin

and

Senior Engineer Ye Peng

College of Software Engineering

Southeast University

September 2019

摘要

区块链技术是下一代云计算的雏形，有望像互联网一样彻底重塑人类社会活动形态，并实现从目前的信息互联网向价值互联网的转变。共识算法是区块链技术的核心，使得区块链可以在无需信任单个节点的情况下构建一个去中心化的可信任系统。共识算法可以直接影响区块链性能的好坏，比特币之所以如此成功也与工作量证明（Proof of Work, PoW）的巧妙设计密不可分。

然而，当前主流的共识算法都有明显缺陷。比特币所使用的 PoW 在共识过程中由于算力竞争造成的资源浪费饱受研究者诟病。实用拜占庭容错机制（Practical Byzantine Fault Tolerance, PBFT）适合应用于联盟链，但其违背去中心化的 C/S 架构以及不支持共识节点的动态加入与退出都是有待解决的问题。Raft 实现复杂且没有拜占庭容错能力，难以在公有链与联盟链中应用。

由于比特币、以太坊等应用每秒交易数量偏小，近几年各种共识算法层出不穷，越来越多共识算法设计由部分节点参与共识从而提升确认交易速度，这就需要保证产生部分节点过程的公平性。可验证随机函数（Verifiable Random Function, VRF）可以生成随机数，且能验证这个随机数确实是由某人生成的，能够作为公平地生成共识节点的解决方案。

本文提出一种基于 VRF 的拜占庭容错共识算法 RBFT。主要工作如下：

1. 在以太坊的衍生应用 Quorum 上对经典共识算法进行了性能实验，通过分析实验结果总结出了改进方向；
2. 对 PBFT 进行了多项改进，主要有节点增减动态化、替换为 P2P 网络模型，还有一些改进以加速共识流程；
3. 算法设计中引入 VRF 与 PoS, VRF 带来了高扩展性以及公平性, PoS 提升了 RBFT 的公平性与安全性
4. 将 RBFT 实现在 Quorum 中，得以直接应用于企业联盟链。经过实验评估得到 RBFT 的实验结果，相比 PBFT、Raft、PoW 等算法都有明显的优势，尤其适合于联盟链场景。

关键字： 区块链，可验证随机函数，拜占庭容错，以太坊

Abstract

Blockchain technology is the prototype of the next generation of cloud computing. It is expected to completely reshape human social activities like the Internet and realize the transition from the current information Internet to the value Internet. The consensus algorithm is the core of the blockchain technology, enabling the blockchain to build a decentralized trusted system without trusting a single node. The consensus algorithm can directly affect the performance of the blockchain, and the success of Bitcoin is inseparable from the clever design of Proof of Work (PoW).

However, all of the current mainstream consensus algorithms have obvious defects. The PoW, which is used by Bitcoin, has been criticized by researchers for its waste of resources caused by computing power competition in the consensus process. The Practical Byzantine Fault Tolerance (PBFT) is suitable for the Consortium Blockchain, but its C/S architecture against decentralization and lack of support for the dynamic joining and exiting of the consensus nodes are both issues to be solved. Implementation of Raft is complex and it has no Byzantine fault tolerance, so that it is difficult to apply it to the Public Blockchain and the Consortium Blockchain.

Due to the small number of transactions per second, such as Bitcoin and Ethereum, various consensus algorithms have emerged in recent years. More and more consensus algorithms are designed to participate in consensus by some nodes to improve the transaction speed, and this requires ensuring the fairness of the process of generating partial nodes. Verifiable Random Function (VRF) can generate a random number and can verify that the random number is indeed generated by someone, which can be used as a solution to generate consensus nodes fairly.

A VRF-based Byzantine fault-tolerant consensus algorithm RBFT is proposed in this thesis. The main contribution are as follows:

1. In the Ethereum derivative application Quorum, the performance experiment of the classical consensus algorithm was carried out, and the improvement direction was summarized by analyzing the experimental results;
2. A number of improvements have been made to PBFT, including node addition and subtraction dynamization, replacement to P2P network model, and other improvements to accelerate the consensus process;
3. It introduces VRF for node lottery and provides verification capability. Combined with PoS, it improves security and fairness;
4. Implement RBFT in Quorum and directly applied to the corporate Consortium Blockchain. The experimental results have obtained the actual results of RBFT, and have obvious advantages in performance, scalability and security compared to PBFT, Raft, PoW and other algorithms.

Keywords: Blockchain, Verifiable Random Function, Byzantine Fault Tolerance, Ethereum

目录

摘要	I
Abstract	II
目录	1
缩略词表	V
第一章 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	1
1.2.1 崩溃容错共识算法	2
1.2.2 拜占庭容错共识算法	3
1.3 论文研究内容	6
1.4 论文组织结构	6
第二章 区块链技术理论	7
2.1 区块链架构	7
2.1.1 数据层	7
2.1.2 网络层	10
2.1.3 共识层	12
2.1.4 合约层	13
2.2 以太坊相关技术	13
2.2.1 区块字段	13
2.2.2 账户余额模型	14
2.2.3 交易	15
第三章 问题描述与改进分析	16
3.1 拜占庭容错共识算法	16
3.1.1 拜占庭将军问题	16
3.1.2 PBFT 相关研究	16
3.2 经典共识算法实验	18
3.2.1 PBFT	19
3.2.2 Raft	21
3.2.3 PoW	22
3.3 实验分析	23
3.3.1 实验对比结果	23
3.3.2 改进目标	24
第四章 RBFT 算法设计与实现	26
4.1 算法改进	26
4.1.1 选取共识节点的改进	26
4.1.2 PBFT 共识流程的改进	29
4.2 算法实现	29
4.2.1 算法流程设计	29
4.2.2 关键模块实现	37

第五章 实验与评估	43
5.1 实验环境.....	43
5.2 性能实验.....	43
5.2.1 TPS 实验	43
5.2.2 共识确认时间实验	46
5.3 拜占庭容错实验.....	47
5.4 实验结果分析.....	48
第六章 总结与展望	49
6.1 总结.....	49
6.2 展望.....	49
致谢	51
参考文献	52

缩略词表

缩略词	英文全称	中文全称
PoW	Proof of Work	工作量证明
PBFT	Practical Byzantine Fault Tolerance	实用拜占庭容错机制
VRF	Verifiable Random Function	可验证随机函数
RBFT	Random Byzantine Fault Tolerance	随机拜占庭容错机制
PoS	Proof of Stake	权益证明
CFT	Crash Fault Tolerance	崩溃容错
BFT	Byzantine Fault Tolerance	拜占庭容错
dBFT	delegated Byzantine Fault Tolerance	授权拜占庭容错机制
PPC	Peercoin	点点币
TPS	Transaction Per Second	每秒交易数
DPoS	Delegated Proof-of-Stake	授权股份证明算法
ECC	Elliptic curve cryptography	椭圆曲线密码学
ECDSA	Elliptic Curve Digital Signature Algorithm	椭圆曲线数字签名算法
DHT	Distributed Hash Table	分布式哈希表
RLP	Recursive Length Prefix	递归长度前缀
UTXO	Unspent Transaction Output	未花费的交易输出
EVM	Ethereum Virtual Machine	以太坊虚拟机

第一章 绪论

1.1 研究背景

区块链技术是下一代云计算的雏形,有望像互联网一样彻底重塑人类社会活动形态,并实现从目前的信息互联网向价值互联网的转变。与传统中心机构的信用背书机制不同的是,区块链形成的是软件定义的信用,这标志着中心化的国家信用向去中心化的算法信用的根本性变革。区块链技术是由 P2P 网络中互相平等的节点共同参与,每个节点都可以视为一个数据库,通过共识算法确保各个节点具有分布式一致性,在此基础上维护一条由众多区块构成的链。在有币区块链中,一般通过制定的激励机制使得所有节点倾向于参与区块的生成、验证过程,一般按照实际的工作量进行 token 的分配。

共识算法是区块链技术的核心^[1]。它的作用主要在于公平地选出指定的节点把交易打包成新区块,并将区块加入区块链中。共识算法使得区块链可以在无需信任单个节点的情况下构建一个去中心化的可信任系统,并且可以直接影响区块链性能的好坏,比特币之所以如此成功也与工作量证明(Proof of Work, PoW)的巧妙设计密不可分。

然而,当前区块链中主流的共识算法都有比较明显的问题。例如由 Nakamoto 设计的比特币所使用的 PoW 虽然解决了拜占庭将军问题并保证了共识的安全性,但在共识过程中由于强大算力造成的资源浪费以及交易确认时间长达 10 分钟饱受研究者诟病^[2]。权益证明(Proof of Stake, PoS)减少了算力竞争的资源浪费,但设计合理的奖惩机制比较困难。实用拜占庭容错机制(Practical Byzantine Fault Tolerance, PBFT)因其不需要货币体系,从而非常适合应用于联盟链,但其违背去中心化的 C/S 架构以及不支持共识节点的动态加入与退出都是有待解决的问题。Raft 是崩溃容错共识算法的代表,然而其实现复杂且没有拜占庭容错能力,难以在公有链与联盟链中应用。

由于比特币、以太坊等应用每秒交易数量偏小,近几年各种共识算法层出不穷,越来越多共识算法设计由部分节点参与共识从而提升确认交易速度,这就需要保证产生部分节点过程的安全性与公平性。可验证随机函数(Verifiable Random Function, VRF)可以生成随机哈希,且能验证这个随机哈希确实是由某人生成的,应能作为公平的生成共识节点解决方案。又因联盟链是主流企业落地最多的区块链场景,目前摩根大通、华为、小米等知名公司均有实践。因此在保证共识性能的基础上,基于 VRF 以及拜占庭容错机制改进出一种高效节能、安全、可容错的共识算法对于企业中区块链的落地是很有应用价值的。

1.2 国内外研究现状

共识算法解决的是分布式系统对某个提案(Proposal),大部分节点达成一致

意见的过程。根据能否解决 Lamport 等研究者于 1982 年提出的拜占庭将军问题，共识算法可以分为崩溃容错 (Crash Fault Tolerance, CFT) 和拜占庭容错 (Byzantine Fault Tolerance, BFT) 两类^[3]。分布式系统在单个企业中部署或由受信任的权威机构运作时，BFT 可能被认为是不必要的，因为会对性能和吞吐量造成过度的拖累。在这种情况下，使用 CFT 可能就绰绰有余了，而在多方、分散的场景中，需要更传统的 BFT 算法。

1.2.1 崩溃容错共识算法

CFT 已经存在不少经典的算法，包括 Paxos、Raft 及其变种等。CFT 性能比较好，处理较快，容忍不超过一半的故障节点。

由 Lamport 提出的 Single-Decree Paxos 算法是首次能比较好地解决分布式一致性问题的算法^[4]。实际上该算法在 1990 年就非正式地提出了。该算法基于消息传递且具有高度容错性，在没有恶意节点情况下，可以保证分布式系统中就某个值达成一致。算法包含 3 种角色：Proposer、Acceptor、Learner。提出后由于晦涩难懂，Lamport 又用更易懂的语言形象描述了该算法^[5]。整个算法的过程可以总结为 2 阶段：第一阶段是 Proposer 向半数以上的 Acceptor 发出 prepare 请求，Acceptor 根据已接受过的 prepare 请求反馈给 Proposer；第二阶段是 Proposer 若收到半数以上 Acceptor 的正确响应，就会发送一个 Accept 请求给 Acceptor，如果 Acceptor 未对更加新的 prepare 请求做出响应就会接受该提案。为保证活性，若一个值被选定，其余节点此时作为 Learner 都会更新到这个值。Gafni 等提出 Disk Paxos 这一基于磁盘的版本，得以支持持久化^[6]。只要大多数磁盘可用，即使所有处理器都无法正常工作，也可以保证一致性。随后 Lamport 等提出 Cheap Paxos，其使用 $F+1$ 个主处理节点与 F 个辅处理节点，在稳定期间辅处理节点不参与共识^[7]。以牺牲活性为代价，减少了主处理节点的数量，实际上提高了性能。在确保活性前提下，为降低消息传递时延，Lamport 又提出 Fast Paxos，通过让每个节点发送请求到多个目标节点处，将 Single-Decree Paxos 中的三次消息延迟降低为两次^[8]。在 Google 的分布式锁服务 Chubby^[9]中首次实现了 Paxos 协议，随后 Chandra 等提出的 Multi-Paxos 则将 Single-Decree Paxos 中 2 阶段简化为 1 阶段，并且能连续就多个值达成一致^[10]。具体方法是从多个 Proposer 中选出一个 Leader，Leader 直接通知 Acceptor 它将递交编号加 1 的提案，Leader 收到半数以上 Acceptor 的正确响应后，Acceptor 就会直接接受该提案。这种简化不仅提高了效率，算法的实现也相对 Single-Decree Paxos 更为简单了。

由 Ongaro 等提出的 Raft 是对 Multi-Paxos 的改进，其容错性和性能与 Multi-Paxos 基本相同，改进在于 Raft 比 Paxos 更易理解和实施^[11]。其设计思路其实借鉴了另一经典的分布式一致性算法 viewstamped replication^[12]。VR 算法采用主机-备份模式，规定所有数据操作都必须通过主机进行，然后复制到各备份机器以

保证一致性。Raft 借这种方式分解了领导者选举、日志复制和安全方面的考虑，并通过约束减少了不确定性的状态空间。

与目前主流的区块链共识算法相比，CFT 主要面向节点数量有限且相对可信的分布式数据库环境，并且不考虑拜占庭容错问题，即假设系统节点只发生宕机和网络故障等非人为问题，而不考虑恶意节点篡改数据等问题。因此，CFT 在区块链中一般只能用于联盟链或私有链^[13]。

1.2.2 拜占庭容错共识算法

BFT 算法，包括 PBFT 为代表的确定性算法、PoW 为代表的概率算法等。确定性算法达成共识结果后区块链不会产生分叉；而概率算法的共识结果则不够稳定，随着共识轮数增加，通过特定共识算法设计的某种机制，共识区块被推翻的概率越来越小。拜占庭容错算法的性能在消息通信较多的情况下比崩溃容错共识算法更差。

传统的拜占庭容错算法最早的讨论可以追溯到拜占庭将军问题，其规定了节点不仅可能崩溃，还可能作恶，即篡改消息或故意不回复消息等，如何在该条件下快速达成共识是研究者一直研究的课题。

由 Bracha 等提出的一种共识算法在异步网络中解决了拜占庭将军问题，然而它并没有给出工业实现的方案，算法的时间复杂度也过高^[14]。另外该算法不能容忍主节点作恶，意味着系统会因此崩溃。由 Canetti 等提出的拜占庭协议通过运用异步可验证秘密分享技术使得系统弹性好，可以快速从错误状态恢复，使得整个系统运行相对更快^[15]。Malkhi 等提出一种拜占庭错误检测机制，可以发现拜占庭节点，但整个检测系统可能误判因而不夠可靠^[16]。Reiter 提出了可靠的拜占庭容错系统 Rampart，当发现恶意节点后会将其从系统中移除确保系统能达成有效共识，但是容忍拜占庭节点的过程开销很大导致时间复杂度仍是指数级的，这也意味着该系统很难在大型工业项目中应用^[17]。

长期以来，拜占庭问题的解决方案都存在运行过慢的问题，直到 1999 年，Castro 等提出了实用拜占庭容错算法 (Practical Byzantine Fault Tolerance, PBFT) 才得以真正改进^[18]。该算法基于前人工作尤其是 Paxos 相关算法进行了优化，首次将异步条件下拜占庭容错算法的时间复杂度从之前的指数级降低到了多项式级，其可以在恶意节点不超过总数 $1/3$ 的情况下同时保证安全性和活性。PBFT 算法采用 RSA 签名算法、消息验证码和消息摘要确保消息传递过程无法被篡改和破坏。其中性能提升主要来自于消息认证码、消息的优化以及设计了检查点机制。PBFT 用消息认证码代替了 Rampart 使用的公钥签名从而减少了大量开销，检查点机制配合垃圾回收得以有序地丢弃不必要的旧消息。算法整体的基本过程如下：首先，通过轮换或随机算法选出某个节点为主节点，此后只要主节点不切换，则称为一个视图 (View)。主节点广播过程包括三个阶段的处理：预准备 (Pre-

Prepare)、准备 (Prepare) 和提交 (Commit)。预准备和准备阶段确保在同一个视图内请求发送的顺序正确; 准备和提交阶段则确保在不同视图之间的确认请求是会依序执行的。PBFT 因其性能良好已经在工业界得到广泛应用, 然而在节点过多时, PBFT 的性能会大大下降。Copeland 等提出在 Raft 算法基础上与 PBFT 结合从而引入了拜占庭容错性的 Tangaroa 算法, 兼顾了可实现性和鲁棒性^[19]。其大致流程与 Raft 相近, 只是在提交阶段采用了 PBFT 的流程以及在保证消息安全上使用了消息签名, 额外加入了 client 可改变主节点这一机制。中国区块链社区 NEO 提出一种改进的拜占庭容错算法 dBFT, 该算法在 PBFT 的基础上借鉴了 PoS 设计思路, 首先按照节点的权益来选出记账人, 然后记账人之间通过拜占庭容错算法来达成共识。该算法改进了 PoW 和 PoS 缺乏最终一致性的问题, 使得区块链能够适用于金融场景。Gilad 等提出基于密码学方法的拜占庭容错算法, 它针对共识算法在公有链情况下性能不佳的问题进行了改进^[20]。主要思想是先选出少量节点, 然后再从这部分节点中随机选取主节点, 避免全网直接进行共识, 将拜占庭算法扩展到了支持较大规模的应用场景, 同时保持较好的性能。

近年来确定性的拜占庭容错算法随着对 PBFT 的改进从而可以应用在公有链, 由于其没有分叉的特性, 可以应用于要求快速确认的金融领域。同时由于共识群体少, 所以需要更注重共识算法的合理性。

概率性的拜占庭容错算法最早来源于 PoW, 它是比特币所使用的共识算法。PoW 思想最早由 Dwork 等于 1993 年提出^[21]。Dwork 应用该思想, 解决了垃圾邮件问题。主要思路为通过要求邮件发送方贡献一定算力, 发送前必须先计算出某个数学难题, 从而抬高了发送垃圾邮件的成本。1997 年, 英国密码学家 Back 独立地提出了用于哈希现金 (HashCash) 的工作量证明机制^[22], 并于 2002 年正式公开其实现。哈希现金目的同样是解决垃圾邮件, 发送方需要将计算出的有至少 20 个前置 0 的哈希值附在消息头作为识别戳记, 且该哈希值包含收件地址、发送日期与盐值, 这样就可以筛选出垃圾邮件。1999 年, Jakobsson 等正式提出了工作量证明概念^[23]。2008 年 10 月, 中本聪发表了比特币白皮书。比特币优秀的共识机制激发了众多机构对于区块链共识算法的研究热情。前文提到的崩溃容错共识算法与确定性拜占庭容错共识算法一般用于节点可信度较高的场景, 也就是联盟链与私有链。在公有链场景则出现了许多概率性拜占庭容错共识算法, 如比特币使用的 PoW。通过 PoW, 比特币确保所有节点的分布式一致性, 该算法参考了 HashCash 的实现方式, 主要思想是通过所有共识节点求解哈希难题产生的算力竞争来保证数据在节点中的分布式一致性以及共识过程的安全性。每个节点需要计算符合当前难度要求且容易验证正确性的 SHA256 数学难题, 最快计算出并广播消息得到大多数节点认可的节点将成为本次的记账节点或称为主节点, 可以将当前交易打包至下一个区块, 区块上链后可以得到一些比特币作为激励。比特币采用工作量证明机制得以较好地实现了商业价值, 使得比特币作为虚拟货

币同样具有货币价值流通的功能,且其优秀的安全性与去中心化程度吸引了全球的投资者,催生了一大批共识算法的产生。不过 PoW 算法也有一些显著的缺陷,比如其强大算力造成的资源浪费(主要是电力消耗)历来为人们所诟病,而且长达 10 分钟的交易确认时间使其相对不适合小额交易的商业应用。

PoW 共识算法的改进目标主要是实现比特币扩容或者降低其能耗。BitcoinNG 对于比特币做出改进,提出划分时间段 slot,在每个 slot 中负责打包区块的 Leader 不同^[24]。共识算法通过设计不同的区块类型从而解决了比特币的扩容问题。关键区块可以给出 Leader,采用比特币共识算法生成。微型区块用来存储交易,由 Leader 打包交易得到。这种算法无需改变区块大小,选出的多个 Leader 可以打包多个微区块得以扩容。ByzCoin 共识算法是具有扩展能力的 BFT 算法,同样是分离了选主过程和交易验证过程。该算法可保证系统有较强的分布式一致性,同时有极高的吞吐量和交易时延^[25]。Elastico 共识算法是应用分片的经典算法。分片提高了整个系统的扩展性,原因是将共识节点网络安全地划分为可以同时处理不同的交易集的许多个分片^[26]。Elastico 是第一个能够安全地做到 BFT 的分片协议。2017 年, Kogias 等结合了 ByzCoin 和 Elastico 这两个共识算法,提出了新的 BFT 协议 ByzCoinX^[27]。ByzCoinX 与 Elastico 一样应用了分片优化了性能,并且还能对区块进行集体签名且减小区块大小,是第一种无需舍弃安全性与去中心化程度就能提高扩展能力的共识算法。

PoS 共识算法在 2011 年提出。PoS 主要解决了比特币资源消耗的问题,通过权益值最高的节点进行打包,无需节点间竞争算力。权益值通过节点持有代币的数量与时间计算得到,该算法由点点币(Peercoin, PPC)实现^[28]。PPC 初期仍然采用 PoW 作为过渡,运行一段时间后使用 PoS 进行共识,使用 PoS 之后可以快速完成共识。2016 年提出的 HoneyBadger 共识算法是首个能得到实际应用的异步 BFT 协议,该算法可以在异步条件下提供活性^[29]。该共识提供了拥有渐进有效性的原子广播协议,能够在数百个节点的情况下达到 10000 以上的 TPS。Ouroboros 共识算法在 2017 年提出,该算法在 PoS 的基础上提供完整的安全性证明,利用博弈论的知识构建了自洽的激励机制,可防止区块截留和自私挖矿等策略性攻击行为^[30]。

Larimer 在比特币白皮书中提出了授权股份证明算法(Delegated Proof-of-Stake, DPoS)^[31],DPoS 的运作过程是由利益相关者也就是代币的持有节点进行投票,通过选举规则得到打包节点,之后区块的打包节点会被随机分配,在一定时间之内打包节点自行决定是否出块,选举规则类似民主选举,每个节点都可以将其持有的股份权益作为选票授予一个代表,获得票数多且愿意成为代表的某些节点将被记录,按照既定的时间表轮流对交易进行打包结算、并且打包生成新区块。其能够很好地解决资源浪费问题,矿池造成的中心化的问题,还能够弥补 PoS 中拥有记账权益的参与者未必希望参与记账的缺点。

1.3 论文研究内容

本文主要围绕拜占庭容错共识算法的改进与实现进行研究。根据各类型共识算法的优缺点和应用场景，且由于企业商业场景尤其适合使用联盟链，故本文提出一种与可验证随机函数相结合的确定性拜占庭容错共识算法。该算法在防分叉、保证效率的前提下，用以改进联盟链节点数量偏少的限制以及保证共识过程的安全性、公平性。

本文研究了区块链领域大多数共识算法，着重研究了实用拜占庭容错共识算法。对三个经典共识算法进行了详尽的实验，通过分析实验结果总结出了改进方向。对 PBFT 进行了多项改进，因为 PBFT 本身无法直接应用于区块链中。改进主要有节点增减动态化、替换为 P2P 网络模型，还有一些细节改进以加速共识流程。在算法设计过程中，引入 VRF 与 PoS，VRF 通过密钥生成算法与零知识证明可以做到无交互抽签，并且能控制中签的数量，可作为选取共识节点的合理方式。VRF 带来了高扩展性以及公平性，PoS 提升了 RBFT 的公平性与安全性。最后将 RBFT 实现在类以太坊的 Quorum 中，得以直接应用于企业联盟链。经过实验评估得到 RBFT 的实验结果，相比 PBFT、Raft、PoW 等算法都有明显的优势，与 Quorum 的结合使得本文改进算法尤其适合用于联盟链场景。

1.4 论文组织结构

全文共分为六章，第一章介绍本文的研究背景，着重于国内外研究现状，对所有共识算法进行总结，其中重点介绍 Paxos 及其衍生算法 Raft、PBFT 以及 PoW 这些具有代表性的共识算法。最后介绍本文的主要研究内容。

第二章首先介绍区块链的基础知识，从宏观上给出区块链架构，具体从数据层、网络层、共识层与合约层这四个与本文研究有重要关联的分层展开研究，以此作为研究的基础。还介绍区块链的重要应用以太坊，以太坊是改进算法 RBFT 实现的基础，其中的区块字段与交易模型构成算法设计的基础。

第三章对拜占庭容错共识算法的改进进行研究，首先研究核心的拜占庭将军问题以及经典的拜占庭容错共识算法 PBFT。之后提出 5 个评估指标，对 PBFT、Raft 与 PoW 三种经典共识算法进行实验，分析得出共识算法改进方向。

第四章给出 RBFT 的算法改进思路，重点介绍 VRF 的引入对于 RBFT 的提升，并对 VRF 的应用以及共识流程进行详细的算法设计。最后介绍 RBFT 的实现，给出关键代码。

第五章从性能、扩展性与安全性的角度对 RBFT 进行实验与评估，与经典共识算法的实验结果对比，验证 RBFT 相比其它算法具有高性能、高扩展性以及良好的安全性等优势。

第六章总结本文的研究工作，介绍本文的主要创新点，并对未来研究方向进行展望。

第二章 区块链技术理论

本章将介绍区块链的技术原理，第一部分介绍本文主要用到的区块链基本架构，首先介绍数据层技术原理理解区块链底层结构，然后分析网络层，再分析本文主要研究的共识层，最后论述合约层，从而对区块链进行全面的理解。再对以太坊相关的技术做出详细阐述，从而作为理解改进算法实现的基础。

2.1 区块链架构

根据袁勇及其研究，区块链从狭义角度是去中心化系统各节点共享的数据账本^[1]。在宏观角度可以分为数据层、网络层、共识层、激励层、合约层与应用层。由于本文主要研究共识算法，基本不涉及激励层与应用层，所以本节将从数据层、网络层、共识层以及合约层的技术原理展开论述，具体如图 2-1 所示。

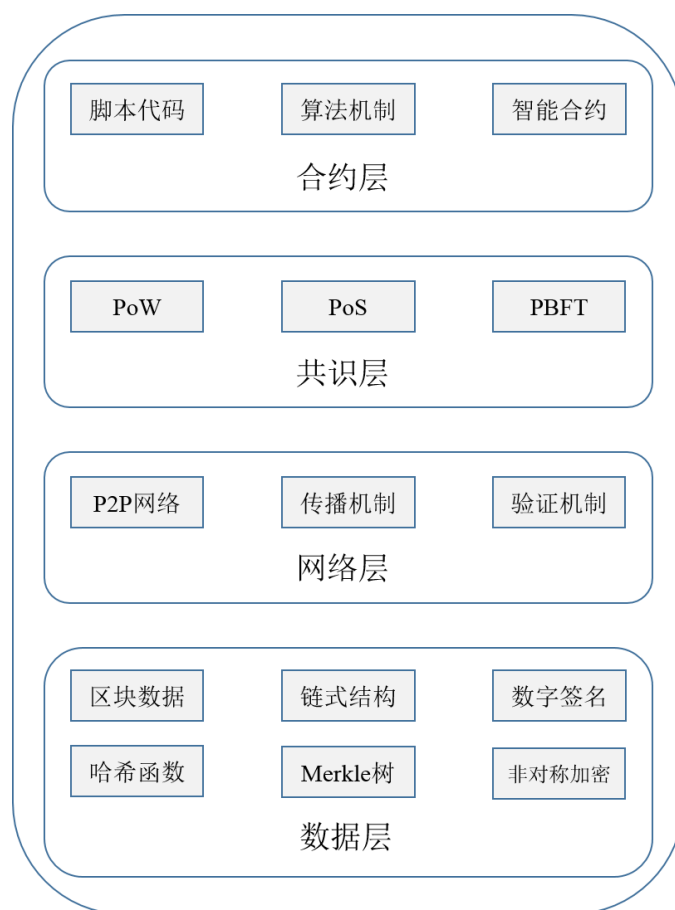


图 2-1 区块链基本架构

2.1.1 数据层

区块链最大的特点是去中心化，在去中心化的条件下需要软件定义的信用系统保证整个系统的运行机制是可信的，数据层就是最基础的保证，通过数字签名、时间戳、Merkle 树等技术为区块链提供算法信用。

2.1.1.1 哈希函数

哈希函数是一种对于任意长度数据 m 提取摘要 $hash$ 的方法，生成的摘要字

字符串由字母与数字构成，其数据量大小相对于原数据来说减小了很多。

$$\text{hash} = H(m) \quad (2.1)$$

哈希函数的性质：

- 1) 单向性：对任意哈希值 h ，要找到一个明文 m 与之对应，即 $h=H(m)$ ，在计算上不可行。
- 2) 弱抗碰撞性：给定明文 m_1 ，要找到另一明文 m_2 ，具有相同哈希值，即 $h(m_1)=h(m_2)$ ，在计算上不可行。
- 3) 强抗碰撞性：要找到任意一对具有相同哈希值的明文，即 (m_1, m_2) ， $h(m_1)=h(m_2)$ ，在计算上不可行。

哈希函数因其性质在区块链中应用广泛，例如在区块数据、Merkle 树的构建、数字签名中均有应用。在区块数据中，每个区块都有标识自身的哈希值，除了创世块，剩余区块都有指向前一个区块的哈希，这样大大减小了存储空间。主流应用中，比特币使用了 SHA2-256。相对于 SHA1，SHA2 只是扩展了哈希的字节数而已。在之前的研究中 SHA1 已经被破解，在未来更大算力情况下，SHA2 也可能被破解。以太坊的哈希算法采用的是全新的 SHA3-256，因为用了新的 Keccak 算法，SHA3 在相同字节长度条件下比 SHA2 更安全。

2.1.1.2 区块结构

区块链的链式结构体现在每个新生成的区块将会通过哈希指针链接到前一个区块上，在一段时间之内的交易会被打包交易者写入当前区块中，通过标记时间戳记录区块产生的先后顺序，从块号为 0 的创世块到最新打包的区块就构成了一条最长的主链，如图 2-2 所示。如果有两个矿工几乎同时把新区块链接到主链上，就会产生分叉现象，此时两个矿工都认为自己打包成功。如何解决分叉在众多区块链应用中有不同的解决方案，以以太坊为例，产生分叉后，下一个区块的打包者会选择两个矿工的某一条链作为主链并链接前一区块，如果该链得到共识节点的认可，即可说明这条链是主链，没被选中的区块称为叔块，也可获得一定的代币激励。总而言之，分叉的处理能力也可以一定程度上衡量区块链系统的好坏。

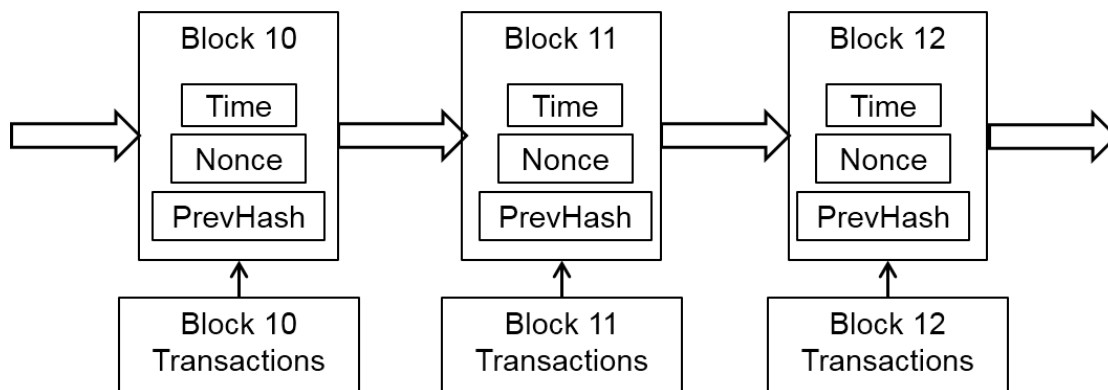


图 2-2 链式结构

每个区块通过哈希值链接上一个区块形成类似链表的结构,通过时间戳作为区块的存在性证明,防止伪造区块,为溯源提供了技术支持。

主流区块链应用中,所有的交易存储在区块中需要消耗节点巨大的存储空间,一个新加入网络的节点在与其它节点同步区块时网络传输量同样巨大,在 2018 年以太坊的全节点存储数据量已经超过 1TB,与全节点对应的只存储区块头的节点称为轻节点,轻节点可以方便地同步区块数据并且能确保这些数据是正确的。保证轻节点的交易有效性通过 Merkle Tree 这种树状结构,以比特币为例, Merkle Tree 是由交易生成的哈希值构成的二叉树, Merkle Tree 的根由所有结点两两进行递归的哈希运算生成,如图 2-3 所示,当篡改某笔交易时,该交易生成的哈希会一层一层向上引起上层结点哈希值的改动直至修改根结点哈希,因此该区块最终会被认定为另一个区块,也就脱离了主链,无法通过全网共识节点的验证。通过 Merkle Tree 这种巧妙的方式,既可以提升区块链的运行效率,还可以让轻节点在没有下载完整区块数据的情况下也能正确验证某笔交易的正确性。

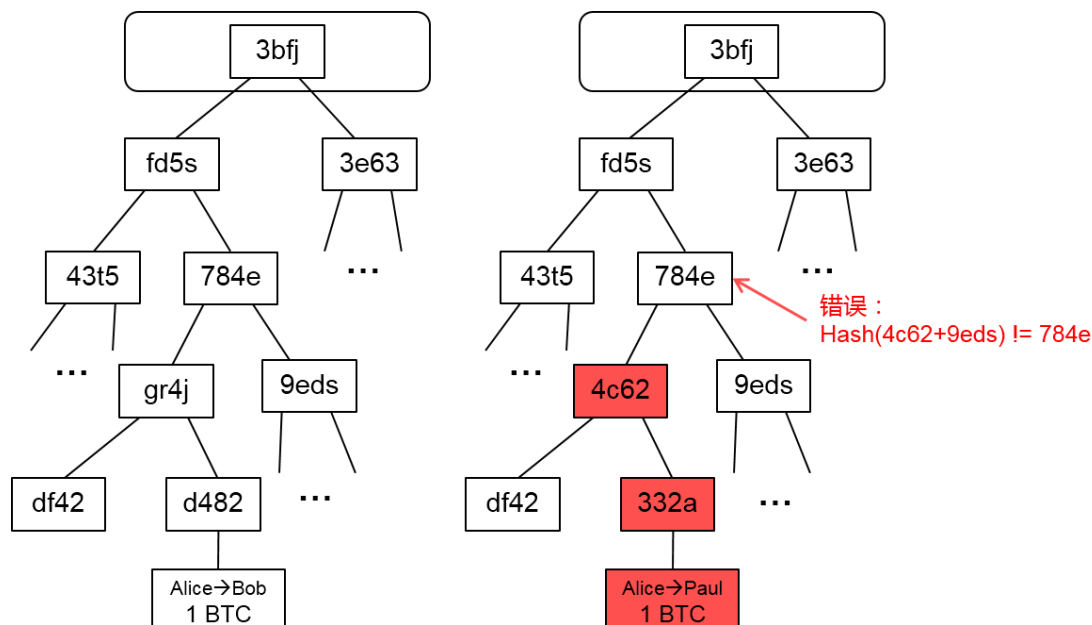


图 2-3 Merkle Tree

2.1.1.3 数字签名

数字签名技术相当于签章,消息接收者可以凭借消息上所附加的某种东西,确认发信人的身份。数字签名有两种实现方式:第一种对消息整体进行签名,第二种对消息摘要进行签名。这两种方式的效果是一样的,然而由于对消息整体进行加密签名十分费时,故一般实际应用第二种方式进行数字签名,流程如图 2-4 所示。数字签名方案可用于提供如下基本密码学服务:消息完整性(确保消息不被以非授权或未知方式改变),消息源可信(确保消息源和声明相同)以及抗抵赖性(确保成员不能否认先前的行为和委托)^[32]。

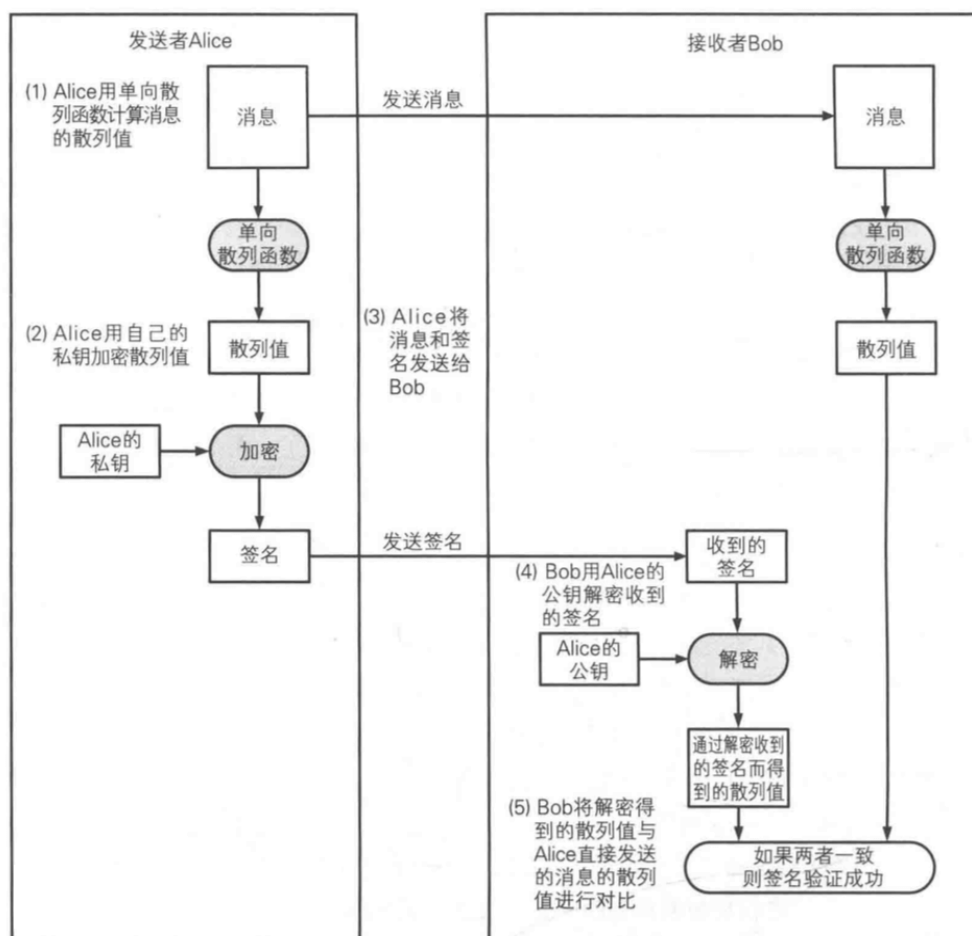


图 2-4 数字签名的实现

区块链中一般采用基于椭圆曲线密码学（Elliptic curve cryptography, ECC）的数字签名：椭圆曲线数字签名算法（Elliptic Curve Digital Signature Algorithm, ECDSA）。ECC 是由 Neal Koblitz^[33]和 Victor Miller^[34]提出的，可看作是传统离散对数密码系统的椭圆曲线对等表示。椭圆曲线系统中，每个密钥位的安全性都比传统的系统强得多，该系统的优点是参数少、速度快以及密钥和证书都较小^[35]。这些优点在处理能力、存储空间、带宽和能源受限的环境中尤其重要，因此 ECDSA 适合应用在区块链系统中。在进行每笔交易时，发送方会进行数字签名标记该笔交易来自于自己。产生新区块时，打包者会在区块上附上自己的数字签名。另外，ECDSA 的公钥经过哈希生成的值称为地址，区块链地址可用于账户的唯一标识。

2.1.2 网络层

P2P 并不是新颖的技术，但因其满足区块链去中心化的需要得以广泛应用于各大区块链系统中。在区块链中，所有节点地位都是对等的，每两个节点可以进行交易而不需要第三方节点监管，且随时可以自行选择退出网络，因此 P2P 协议尤其适合。不同于传统 C/S 架构，每个节点既可作为服务端提供服务，也可作为

客户端请求服务，且都具有路由、存储等能力。只要节点没有全部失效，区块链服务与数据就能正常保持。

比特币和以太坊的 P2P 协议基于 TCP 协议实现，Hyperledger Fabric 的 P2P 协议则基于 HTTP/2 协议实现。以以太坊为例，以太坊采用的是全分布式结构化拓扑的 P2P 网络架构，节点之间通过 TCP 协议建立连接以及传输消息，节点发现则是通过 UDP 和分布式哈希表（Distributed Hash Table, DHT）实现，图 2-5 展示了以太坊的 Go 语言实现版本的基本通信流程。DHT 负责存放当下活跃的节点的 URL，当某个节点 A 需要连接其它节点时，A 每间隔一段时间会向邻居节点 B 通过 UDP 请求 B 的 DHT，B 再通过 UDP 返回 DHT 存放的其它可连接的邻居节点的 URL，这样就完成了节点发现。

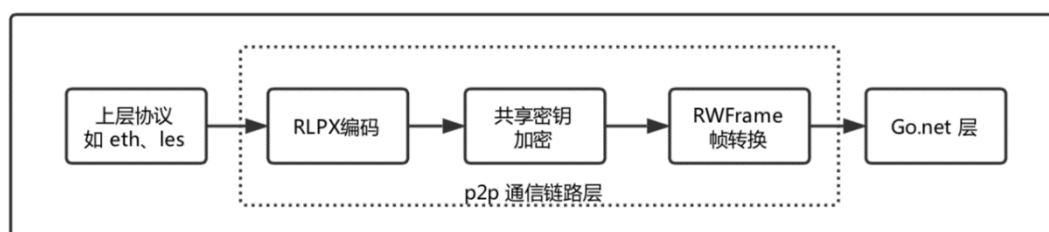


图 2-5 以太坊 P2P 通信

在编码技术上，以太坊采用了递归长度前缀（Recursive Length Prefix, RLP）。RLP 作为轻量级的序列化方式，仅用嵌套的字节数组对结构体编码，而不对整形、浮点型等原子数据类型编码。在 RLP 中，编码对象可视为字符串或是字符串的集合。根据以太坊 wiki，具体编码规则如下：

- 1) 对于[0x00,0x7f]范围内的单个字节，RLP 编码内容就是字节内容本身。
- 2) 否则，如果是一个 0-55 字节长的字符串，则 RLP 编码有一个特别的数值 0x80 加上字符串长度，再加上字符串二进制内容。这样，第一个字节的表达范围为[0x80,0xb7]。
- 3) 如果字符串长度超过 55 个字节，RLP 编码由定值 0xb7 加上字符串长度所占用的字节数，加上字符串长度的编码，加上字符串二进制内容组成。比如，一个长度为 1024 的字符串，将被编码为\xb9\x04\x00 后面再加上字符串内容。第一字节的表达范围是[0xb8,0xbf]。
- 4) 如果列表的内容（它的所有项的组合长度）是 0-55 个字节长，它的 RLP 编码由 0xc0 加上所有的项的 RLP 编码串联起来的长度得到的单个字节，后跟所有的项的 RLP 编码的串联组成。第一字节的范围因此是[0xc0,0xf7]。
- 5) 如果列表的内容超过 55 字节，它的 RLP 编码由 0xc0 加上所有的项的 RLP 编码串联起来的长度的长度得到的单个字节，后跟所有的项的 RLP 编码串联起来的长度的长度，再后跟所有的项的 RLP 编码的串联组成。

第一字节的范围因此是[0xf8,0xff]。

下面给出一些示例：

- 1) 字符串"dog"=[0x83,'d','o','g']
- 2) 列表["cat","dog"]=[0xc8,0x83,'c','a','t',0x83,'d','o','g']
- 3) 空字符串('null')=[0x80]
- 4) 空列表=[0xc0]
- 5) 数字 15('\x0f')=[0x0f]
- 6) 数字 1024('\x04\x00')=[0x82,0x04,0x00]

在区块链网络中，节点时刻监听网络中广播的数据，当接收到邻居节点发来的新交易和新区块时，其首先会验证这些交易和区块是否有效，包括交易中的数字签名、区块中的工作量证明等，只有验证通过的交易和区块才会被处理（新交易被加入正在构建的区块，新区块被链接到区块链）和转发，以防止无效数据的继续传播^[36]。

2.1.3 共识层

共识层有选出区块提案者即主节点、验证交易及主节点的合法性等职责，其作用在于通过所有参与共识过程的节点在互不信任的情况下达成对区块数据的共识。图 2-6 给出了共识层的核心流程。根据选用不同的共识算法，对拜占庭将军问题的解决程度是不一样的，比如前文提到的 Paxos 与 Raft 算法，就无法应对节点作恶行为，有可能破坏共识过程的可靠性。下面叙述共识层的核心流程：

- 1) 接收到由 P2P 网络初次验证的交易数据，各个共识节点开始准备选出主节点。
- 2) 共识节点集在某种共识算法的规则下通过广播相互协商出主节点，一般需要达到一定性能要求。当节点收到一半以上的相同消息答复时，将把该消息内的主节点作为本节点认定的，再广播出去。最后一般由一个节点作为主节点。
- 3) 选出的主节点依据不同链选用的特定规则，附上时间戳与 Merkle 根等区块信息，并将这段时间发生的所有交易打包到一个区块，把这个新区块广播给共识节点集，区块上附有主节点的数字签名，可供共识节点集验证。
- 4) 共识节点集中的节点收到消息后将分别从两个方面验证新区块，首先会验证区块相关信息，如验证区块签名、通过 Merkle Tree 重新推算验证区块头、区块大小验证等；之后验证每笔交易的合法性，包括交易数据量、校验交易手续费、转出方账户余额不能小于交易金额加手续费等。
- 5) 一般一半以上共识节点验证通过后，主节点可进行区块上链，将新区块链接到主链，其它节点可通过 P2P 网络同步主链得到最新区块。

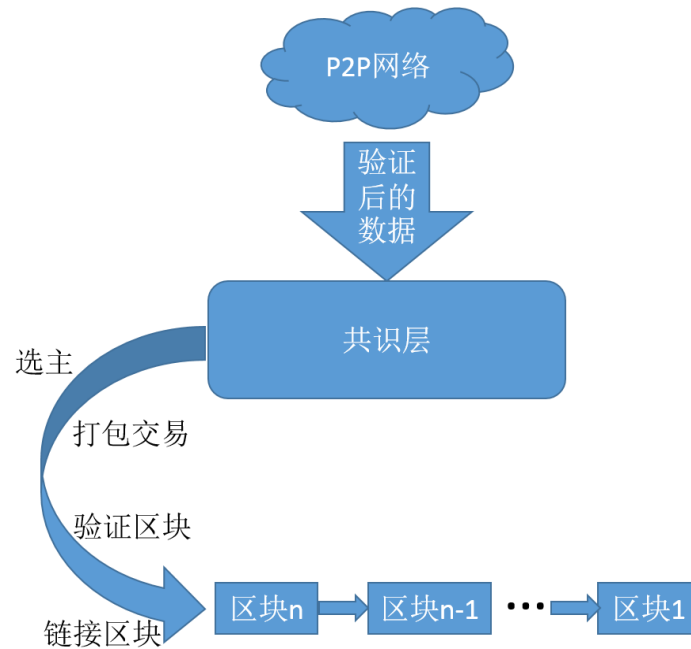


图 2-6 共识层核心流程

2.1.4 合约层

智能合约是区块链又一个重要技术，因区块链具有分布式可信特点，智能合约才得以顺利应用。智能合约是具有状态、由代码构成的程序，当满足触发条件时，合约会自动由执行引擎执行编译后的字节码。合约可以通过变化状态储存交易给予的价值、花费价值、操作数据库进行持久化存储，所以其可以作为区块链中的交易规则，实现算法信用。模型如图 2-7 所示。

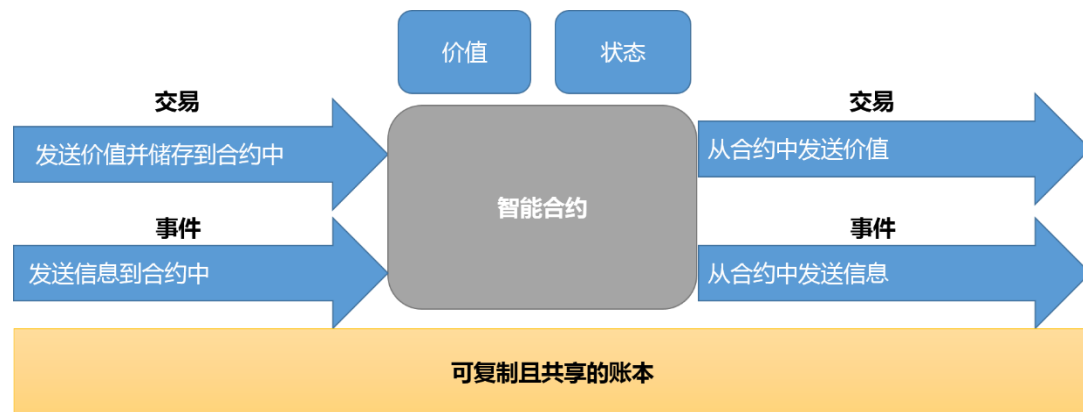


图 2-7 智能合约模型

2.2 以太坊相关技术

本文最终实现基于以太坊，因其作为区块链中已受大量实践检验的流行应用，故在此介绍以太坊的技术细节。

2.2.1 区块字段

以太坊区块字段如表 2-1 所示。其中 gasLimit 的大小决定了区块大小，该字段越大则区块能够包含的交易越多。在以太坊 ethash 共识算法下，difficulty、

mixHash、nonce 等字段提供了工作量证明的能力。

表 2-1 以太坊区块字段

字段	说明
parentHash	父区块头的 Keccak256 位哈希
ommersHash	当前区块的 ommers 列表的 Keccak256 位哈希
beneficiary	地址，用于接收成功挖矿的奖励
stateRoot	所有交易被执行完且区块定稿后的状态树（state trie）根节点的 Keccak256 位哈希
transactionsRoot	由当前区块中所包含的所有交易所组成的树结构（transaction trie）根节点的 Keccak256 位哈希
receiptsRoot	由当前区块中所有交易的收据所组成的树结构（receipt trie）根节点的 Keccak256 位哈希
logsBloom	由当前区块中所有交易的收据数据中的可索引信息（产生日志的地址和日志主题）组成的 Bloom 过滤器
difficulty	当前区块难度水平的纯量值，它可以根据前一个区块的难度水平和时间戳计算得到
number	当前区块的祖先的数量，创世块该值为 0
gasLimit	目前每个区块的 gas 开支上限
gasUsed	当前区块的所有交易所消耗的 gas 统计
timestamp	当前区块初始化时的时间戳
extraData	与当前区块相关的任意字节数据，但必须在 32 字节以内
mixHash	工作量证明用到的 256 位哈希值，用来与 nonce 一起证明当前区块已经承载了足够的计算量
nonce	一个 64 位的值，用来与 mixHash 一起证明当前区块已经承载了足够的计算量

2.2.2 账户余额模型

以太坊的账户余额模型不同于比特币的 UTXO 模型。UTXO 模型中没有账户概念，每笔交易只有已花费与未花费两种状态，未花费之和即为余额。在查找任意地址的余额时，需要遍历所有区块找到该地址未花费的交易，效率不够高。账户余额模型更易理解，每一个地址作为一个账户拥有余额，交易发送前不需遍历所有区块，直接校验账户余额即可。账户模型具体字段如表 2-2 所示。

以太坊账户类型一般分为两种：通过密钥控制的账户、通过代码控制的合约账户。参与人员只能得到前一种账户，与比特币地址类似，通过创建和签名一笔交易向全网发送消息。合约账户是以太坊引入的，合约账户被消息触发后执行存储的代码从而对数据进行读写操作，可作为其它账户的服务提供者。

表 2-2 以太坊账户字段

字段	说明
nonce	作为一笔交易的唯一标记，可用于防止双花攻击
codeHash	本账户的合约代码的哈希
balance	账户余额，单位为 Wei
storageRoot	保存了账户的存储内容的 Merkle Patricia 树的根节点的 256 位哈希值，这个树中保存的是 256 位整数键值的 Keccak 256 位哈希值到 256 位整数值的 RLP 编码的映射

2.2.3 交易

以太坊中的交易检验主要包含了交易格式、签名、nonce 值的校验以及手续费的计算。

燃料 (gas) 是以太坊交易中重要的部分, gasPrice、gas 数量经过计算可作为交易手续费, 每单位 gas 具有价格, 在发送交易方执行交易代码时, 根据交易大小计算 gas 费用。以太坊提供了图灵完备的智能合约平台, 意味着智能合约可以实现循环等复杂逻辑, 而 EVM 若陷入无限循环会使整条链瘫痪。表 2-3 的 gasLimit 字段可以避免 EVM 交易时无限循环, 因为每次执行代码消耗的 gas 累加到达 gasLimit 时就会认定交易失败, 且交易发送者已支付的手续费无法收回。

表 2-3 以太坊交易字段

字段	说明
nonce	由交易发送者发出的交易的数量
gasPrice	为执行这个交易所需要进行的计算步骤消耗的每单位 gas 的价格, 以 Wei 为单位
gasLimit	用于执行这个交易的最大 gas 数量。这个值须在交易开始前设置, 且设定后不能再增加
to	160 位的消息调用接收者地址
value	转移到接收者账户的 Wei 的数量
v, r, s	与交易签名相符的若干数值, 用于确定交易的发送者
init	一个不限制大小的字节数组, 用来指定账户初始化程序的 EVM 代码
data	一个不限制大小的字节数组, 用来指定消息调用的输入数据

第三章 问题描述与改进分析

本章首先研究了拜占庭容错共识算法中核心的拜占庭将军问题以及较好解决了该问题的 PBFT 共识算法。给出了共识算法的实验核心指标,对 PBFT、Raft、PoW 这三个区块链中经典的共识算法进行了实验,最后通过实验结果归纳总结了共识算法的改进目标与具体要求。

3.1 拜占庭容错共识算法

本文对于拜占庭容错共识算法做出改进,故介绍拜占庭将军问题以更好理解拜占庭容错的原理。另外详细介绍了 PBFT,因为 PBFT 是最经典的拜占庭容错共识算法。

3.1.1 拜占庭将军问题

一些来自拜占庭的将军们在不同的地方拥有自己的军队,通过投票来共同决定一起进攻或一起撤退。军队间通过信使进行沟通,每个将军根据收到信使的信息以及自己的选择进行统计即可知道最终决定。问题关键在于将军可能背叛,可以通过不投票、故意投错票、伪造其它将军的信件投假票、杀害别的将军的信使等手段干扰将军们最终达成行动上的一致性。

将上述的问题与计算机关联起来,可以对应到分布式系统。一个将军对应一台服务器,信使可以对应为服务器之间的消息,服务器被攻击者控制后可以做出一系列行为影响整个系统达成分布式一致性。

由此引出拜占庭容错系统:某分布式系统具有 $3f+1$ 台服务器,则集群中拜占庭服务器(即可能带有攻击性的服务器或宕机的服务器)不得超过 f 台。在此条件下,系统响应每个消息请求需满足:

- 1) 对所有正常服务器而言,当输入信息相同,最终产生一致的结果。
- 2) 正常服务器能够正确响应正确的输入,并进行处理计算。

消息请求还需要满足两个指标^[37]:

- 1) 安全性(safety):已经处理的请求具有持久性,不再能被修改。
- 2) 活性(liveness):可以接受并且执行非拜占庭客户端的请求,不会被任何因素影响而导致非拜占庭客户端的请求不能执行。

3.1.2 PBFT 相关研究

当主服务器收到客户端请求时,它会启动一个三阶段协议,以便将请求原子性地广播到其它服务器,这里称为副本。三阶段是预准备、准备和提交阶段。预准备和准备阶段用于对在同一视图中发送的请求进行正确排序,即使提出排序请求的主节点发生故障。准备和提交阶段用于确保提交的请求得到顺序执行。主节点的选取通过取余得到,如公式 3.1 所示。

$$p = v \% R \quad (3.1)$$

其中 p 为副本编号, v 是当前视图编号, R 是副本的总数量。

在预准备阶段, 主节点 p 为请求分配一个序列号, 将一个预准备消息广播到所有副本, 然后将消息附加到其日志中。请求未包含在预准备消息中以保持消息较小。预准备消息可以证明在视图更改中视图 v 中的请求被分配了序列号 n 。此外, 它将请求排序与请求传输到副本这两步进行了解耦。

此时副本接受的消息需满足:

- 1) 签名和消息摘要的正确性;
- 2) 视图的正确性;
- 3) 之前没接受过包含当前视图 v 以及序列号 n 的不同消息摘要;
- 4) 序列号在合理范围, 目的是防止序列号耗尽。

当副本 i 接受了预准备消息, 它就进入了准备阶段, 会广播一个 `prepare` 请求并把预准备请求中的消息 m 以及自己的 `prepare` 请求中的消息加入日志中。前两个阶段保证了非拜占庭副本能在同一个视图内对请求的执行顺序达成共识。`prepare` 请求包含一系列的校验, 若返回 `true` 则进入提交阶段, 广播一个 `commit` 请求, 其它副本比较一下视图以及请求的序列号, 正确即可插入到该副本的日志中。

我们按如下方式定义 `committed` 和 `committed-local`: 当且仅当 `prepared(m, v, n, i)` 对于 $f+1$ 个无故障的副本都为 `true` 时, `committed(m, v, n)` 才为 `true`; 当且仅当 `prepared(m, v, n, i)` 为 `true` 且副本 i 已接受与 `pre-prepare message` 匹配的不同副本的 $2f+1$ 个提交 (可能包括其自己的) 时, `committed-local(m, v, n, i)` 才为 `true`。

提交阶段确保以下性质: 如果对于某些非拜占庭副本 i , `committed-local(m, v, n, i)` 为 `true`, 则 `committed(m, v, n)` 为 `true`。这个性质和视图更改协议共同确保非拜占庭副本在本地提交的请求的序列号上达成一致, 即使它们在每个副本的不同视图中提交。此外, 该性质确保在非拜占庭副本本地提交的任何请求最终将在 $f+1$ 或更多非拜占庭副本处提交。

每个副本 i 在 `committed-local(m, v, n, i)` 为 `true` 之后执行 m 中请求的操作, 并且 i 的状态反映了具有较低序列号的所有请求的顺序执行。这可确保所有非拜占庭副本按照提供安全性所需的相同顺序执行请求。执行请求的操作后, 副本会向客户端发送回复。副本会丢弃时间戳低于它们发送给客户端的最后一个回复中的时间戳的请求, 得以保证一次性语义。

假设节点总数为 a , 在预准备阶段, 主节点广播 `pre-prepare` 消息给其它节点即可, 因此通信次数为 $a-1$; 对于准备阶段, 每个节点如果同意请求后, 都需要向其它节点再广播 `prepare` 消息, 所以总的通信次数为 $a \cdot (a-1)$; 对于提交阶段, 每个节点如果达到 `prepared` 状态后, 都需要向其它节点广播 `commit` 消息, 所以通信次数也为 $a \cdot (a-1)$ 。相加得到总通信次数为 $2n^2 - n - 1$, 因此 PBFT 广播

消息的时间复杂度为 $O(n^2)$ 。

整个过程如图 3-1 所示。

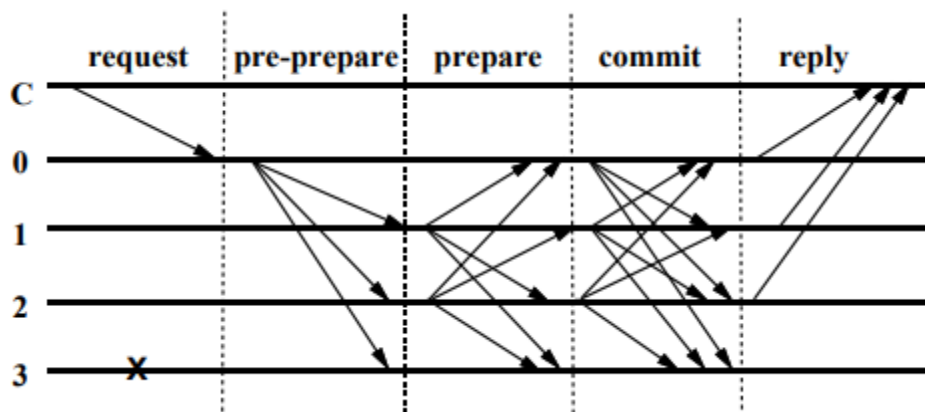


图 3-1 PBFT 流程

3.2 经典共识算法实验

本文将基于摩根大通研发的基于以太坊的企业级分布式账本 Quorum 进行实验。Quorum 由 Go 语言编写，是 go-ethereum 的一个分支，在 go-ethereum 的基础上额外提供联盟链的能力。Quorum 支持隐私保护，并且新增了 Raft 与 PBFT 两种共识机制，这两个共识算法分别代表了私有链以及联盟链的常用共识算法。

本文以交易吞吐量、共识确认时间、共识节点数量、防恶意节点数、资源消耗等作为评估指标，从而得到已有经典共识算法的优缺点以作为之后改进效果的参考。具体评估指标如下：

- 1) 交易吞吐量：本文用每秒交易数（Transaction Per Second, TPS）来表示文中区块链系统的交易吞吐量，该指标可以衡量区块链的共识性能。TPS 通过计算一定时间内的总交易数除以这段时间得出，如公式 3.2 所示。

$$TPS = \frac{TotalTransactions}{t} \quad (3.2)$$

- 2) 共识确认时间：共识确认时间是衡量一个区块链系统中共识算法的执行效率以及消息广播时延的指标。共识确认时间如果相比其它算法较低，代表着交易快速打包出块减少分叉的概率，用户进行交易的体验也会更好。本文共识确认时间指的是交易发出之后直至该交易打包进区块的执行时间，如公式 3.3 所示。

$$T_{cc} = T_{bc} - T_{ts} = T_{TransBroadcast} + T_{Consensus} + T_{BlockBroadcast} \quad (3.3)$$

其中 T_{cc} 为共识确认时间， T_{bc} 为区块打包时间， T_{ts} 为交易发出时间， $T_{TransBroadcast}$ 为交易在广播时产生的消息通讯时间， $T_{Consensus}$ 为运行共识算法代码耗费的时间， $T_{BlockBroadcast}$ 为区块在广播时产生的消息通讯时间；

- 3) 共识节点数量：共识节点数量指参与共识过程的节点的最大数量，通常通过测试保持共识过程高效运行所能容忍的最大节点数量得来，是衡量

去中心化程度的重要指标；

- 4) 拜占庭容错能力：通过防恶意节点数体现。恶意节点即拜占庭节点，可以通过篡改消息、无应答等行为干扰共识算法，具体数量通过增加恶意节点数量直至共识算法崩溃得来；
- 5) 资源消耗：主要指竞争记账权过程中算力的消耗，取决于共识算法的记账机制。

本节采用 8 台服务器，单机环境如表 3-1 所示，其中网络带宽上行带宽与下行带宽均为 100Mbps，足以支撑本文实验。本文采用 Python 编写脚本进行实验。

表 3-1 经典共识算法实验环境

环境名称	说明
操作系统	ubuntu18.04
CPU	Intel Xeon X3220 2.4GHz
内存	16G
网络带宽	100Mbps

3.2.1PBFT

为了尽可能模拟出最高性能，在多次实验之后，在 7 节点构成的 Quorum 联盟链中，分别用单线程、3 个线程、5 个线程、10 个线程、20 个线程、50 个线程与 100 个线程作为制造交易的 worker 同时发送交易，实验结果如图 3-2 所示。

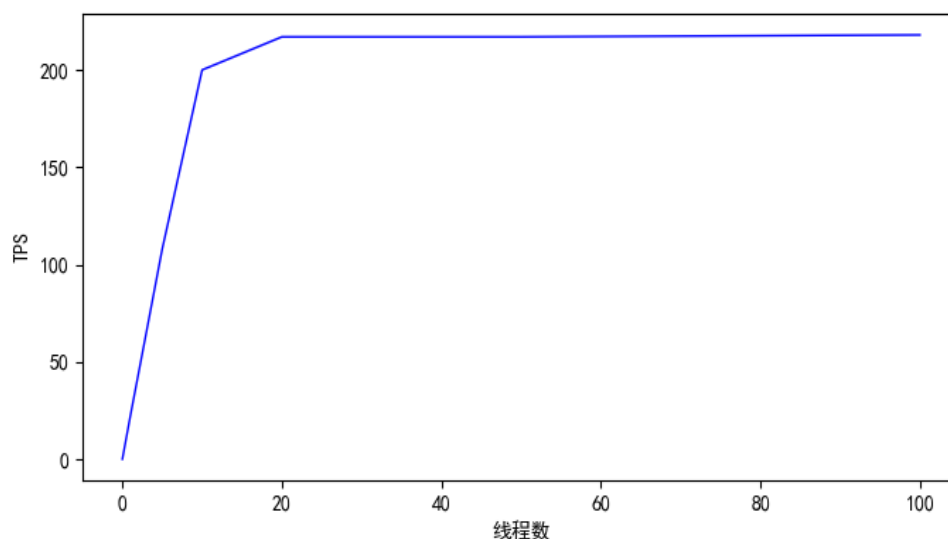


图 3-2 发送交易线程数与 TPS 关系

由图可知在大于 20 个线程同时发送交易时 Quorum 联盟链的 TPS 达到峰值，所以取 30 个线程同时发送交易，并进行多次性能实验。由于区块大小也会影响区块链的性能，而以太坊的区块大小由前文提到的区块字段 gasLimit 进行控制，故在 30 个线程发送交易的条件下修改 gasLimit 进行实验。

Quorum 的默认值为 4.7×10^6 ，在这里取 10^5 ， 3×10^5 ， 10^6 ， 2×10^6 ， 4.7×10^6 ， 10^7 ， 2×10^7 ， 3×10^7 分别进行实验。得到 TPS 的结果如图 3-3。由图可

知 gasLimit 即出块大小对 TPS 有较大影响, gasLimit 在达到 2×10^7 之后基本能够满足需求, TPS 基本无法再增大, 故 gasLimit 选择 2×10^7 。

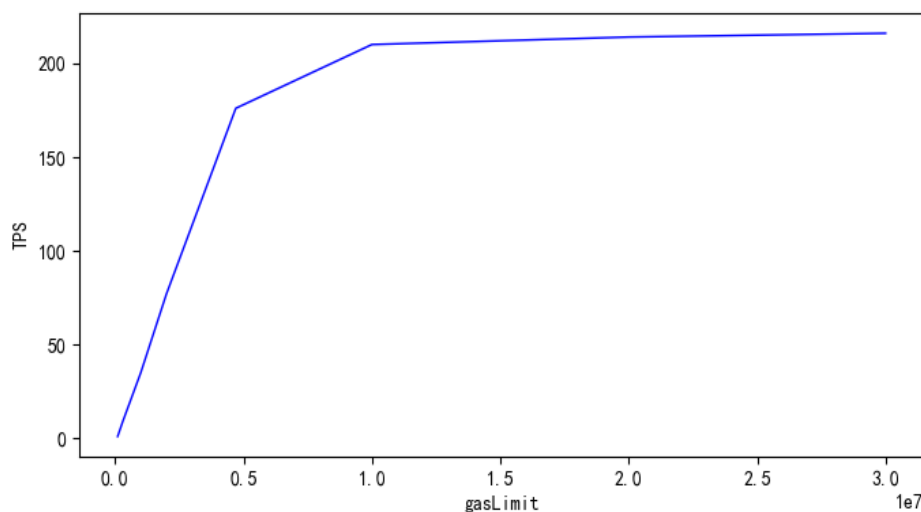


图 3-3 PBFT 中 TPS 与 gasLimit 关系

不失一般性, 本文进行重复实验。取 30 个线程作为 30 个 worker 同时发送交易, gasLimit 取 2×10^7 , 进行 12 次性能实验。

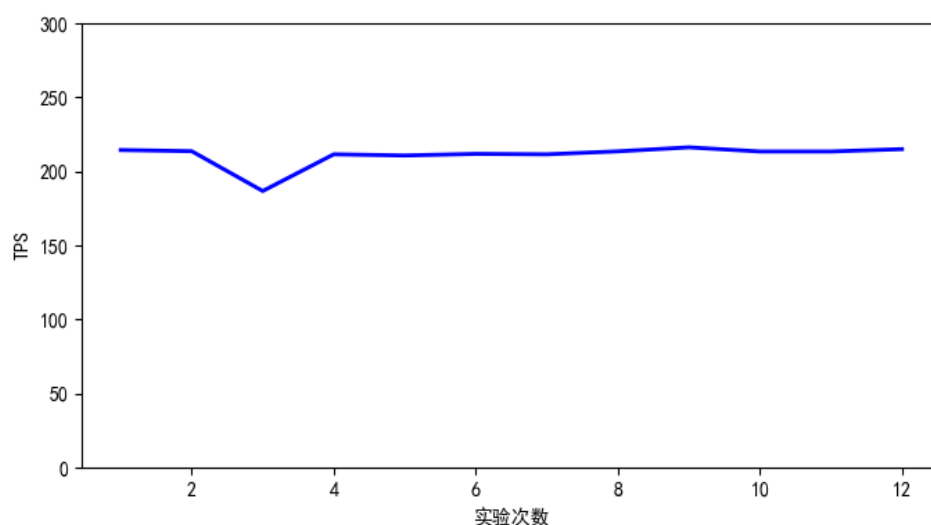


图 3-4 PBFT 的 TPS

如图 3-4 所示, PBFT 在 7 节点情况下 TPS 大概在 210 至 215 之间, 平均 TPS 经过计算为 211 左右。

由 7 节点逐渐增加至 56 个节点, 每次增加 7 个节点并进行 12 次重复实验, TPS 取平均值。平均 TPS 变化如图 3-5 所示。PBFT 在增加节点至 30 以上时性能明显下降, 主要因为 PBFT 算法广播消息的时间复杂度为 $O(n^2)$, 在节点数量增大时, 各节点之间广播消息的时延的开销过大。

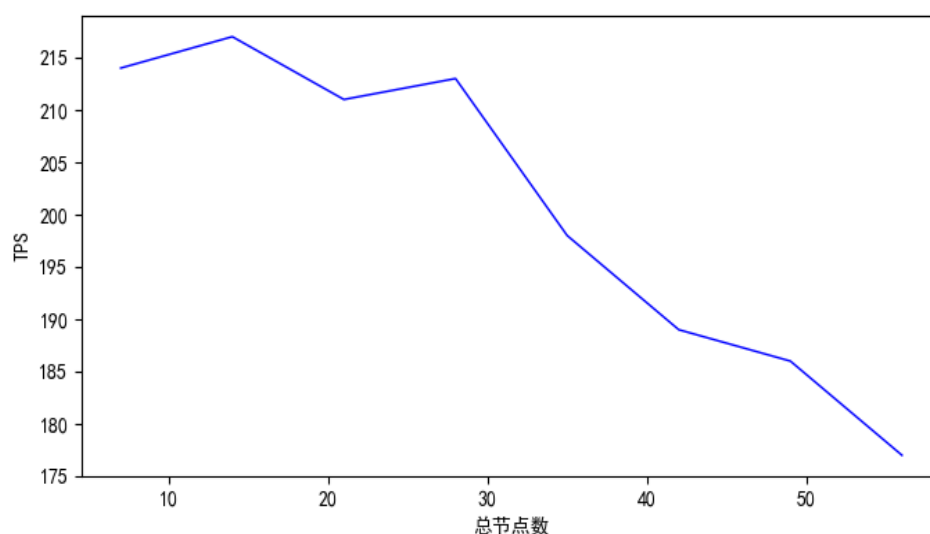


图 3-5 PBFT 节点数量与 TPS 关系

3.2.2Raft

上文提到 Raft 作为 Paxos 的易实现版本被广泛应用于分布式系统，可以作为非拜占庭容错共识算法的代表进行性能实验。

Raft 对于 Paxos 有一些简化，比如只有 leader 可以向其它节点发送日志，在 Paxos 中任何节点都可以，这对于工程实现很有帮助。另外在 Raft 中日志必须连续。在 Quorum 中，leader 与区块打包者为同一节点，一旦节点经过 Raft 的选举算法成为 leader 后，该节点开始进行区块打包操作。不同于 PBFT 的是，Raft 在没有交易发生时不会产生空块，减少了不必要的开销。

由于线程数对 TPS 的影响基本与共识算法无关，所以 Raft 可以继续沿用 30 个线程进行发送交易。gasLimit 依次取 10^5 ， 2×10^5 ， 3×10^5 ， 10^6 ， 2×10^6 ， 4.7×10^6 进行实验，结果如图 3-6 所示。

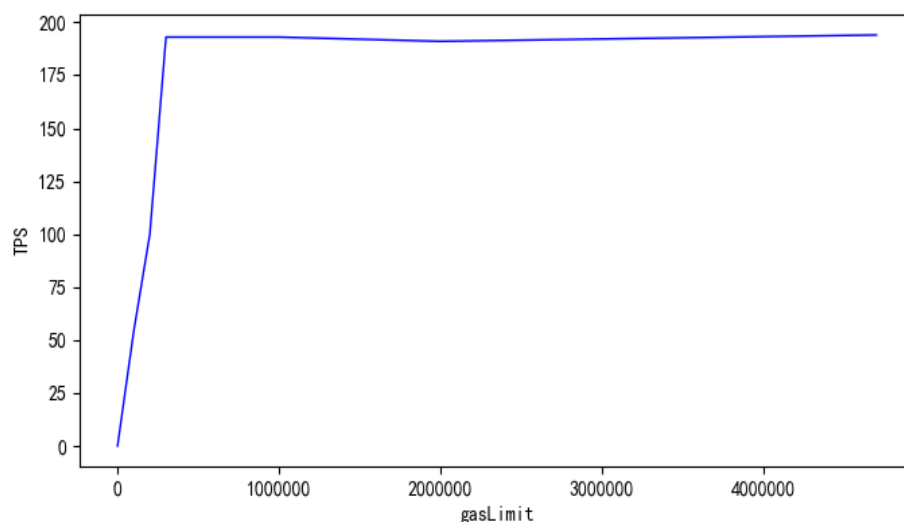


图 3-6 Raft 中 TPS 与 gasLimit 的关系

经过实验, gasLimit 在大于 3×10^5 后 TPS 就达到了峰值, 所以选用 4.7×10^6 的默认值已经能够满足性能实验的要求。

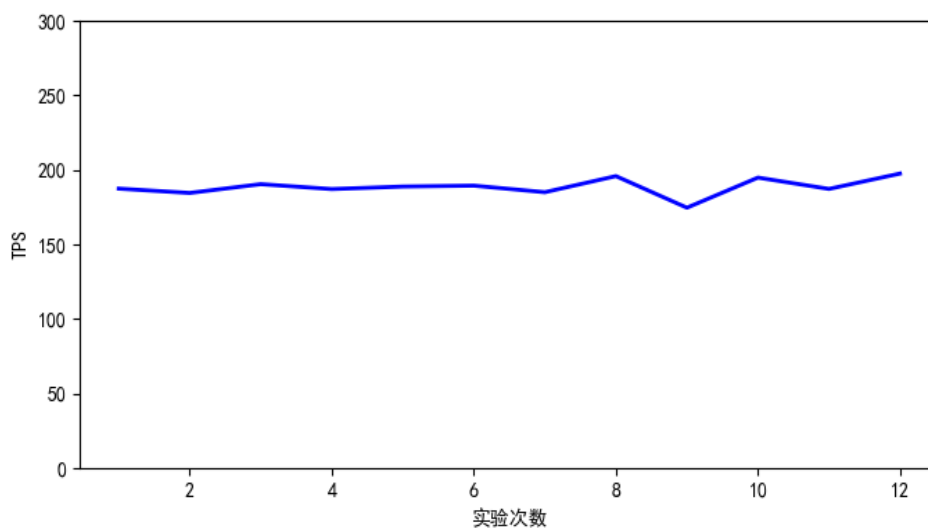


图 3-7 Raft 的 TPS

在经过 12 次对 Raft 的性能实验后, 实验结果如图 3-7 所示, 经过计算, Raft 的平均 TPS 为 188.5 左右, 略低于 PBFT。

同样每次增加 7 个节点直至 56 个节点, 每次进行 12 次重复实验, Raft 的平均 TPS 变化情况如图 3-8 所示。Raft 在增加节点至 30 以上后, 性能同样开始有较明显下降, 不过相比 PBFT 情况更好一些, 因为 Raft 算法广播消息的时间复杂度为 $O(n)$ 。

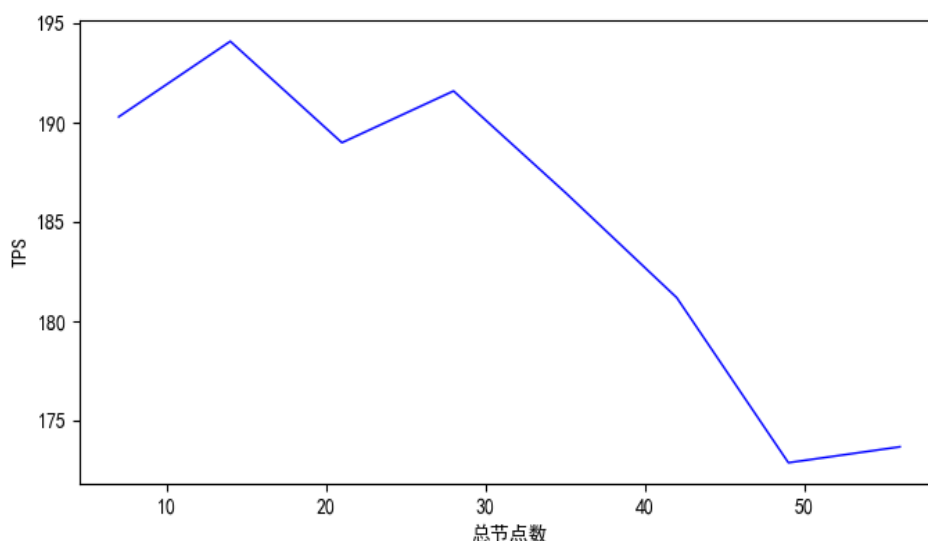


图 3-8 Raft 节点数量与 TPS 关系

3.2.3PoW

PoW 在以太坊中得到应用, 以太坊公有链采用的 Ethash 共识算法的思想即来源于工作量证明。由于 Ethash 算法无法在联盟链上进行部署测试, 故本文引

用以太坊的官方性能数据，如图 3-9 所示。由于以太坊交易队列中一直有未处理完的交易，所以其一直处于性能瓶颈，由图可见以太坊平均每天处理约 6×10^5 笔交易，经过计算，TPS 约为 6.9，远远低于以上两种共识算法。

PoW 的每次共识过程都需全网节点共同计算验证，这是该算法性能低下的根本原因，但其共识节点数量为全部节点，这意味着其去中心化程度是最高的。

Quorum 在以太坊基础上添加了隐私保护，其性能较以太坊理应大幅下降，然而以太坊采用的 Ethash 算法的 TPS 却远低于 Quorum 的 PBFT 与 Raft 共识算法的实验结果。因此，在联盟链中采用 PBFT 与 Raft 具有明显的优势。

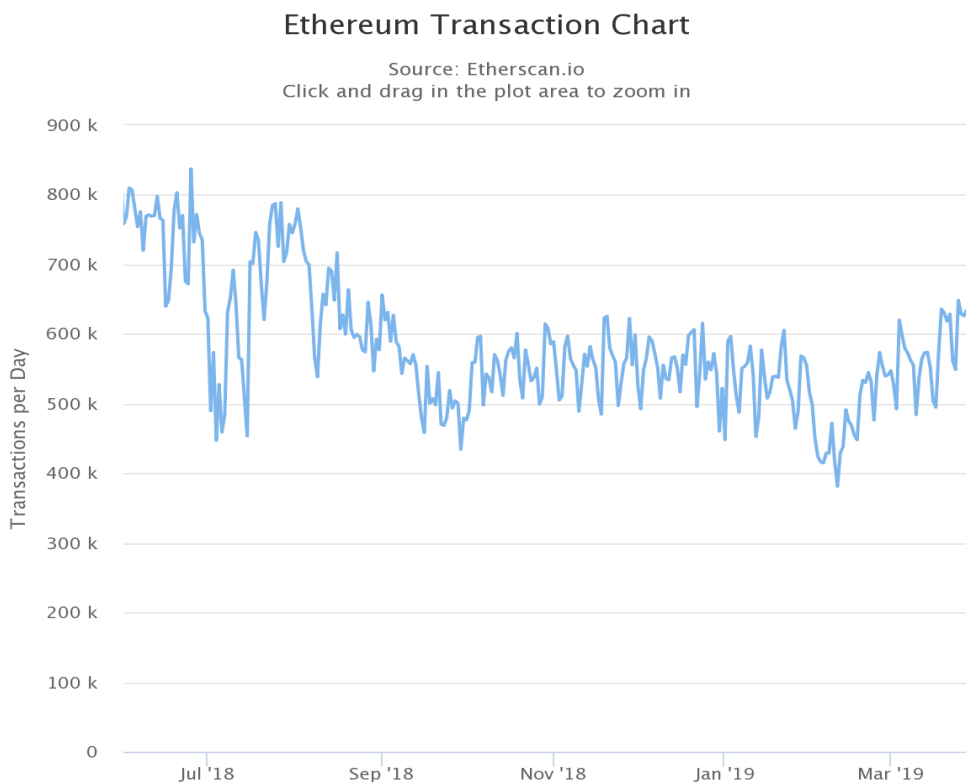


图 3-9 以太坊每日交易数

3.3 实验分析

本节通过经典共识算法的实验结果对比分析，总结出共识算法改进的三个方向：性能、扩展性、安全性，并为之后的改进效果提供参考，可以基于同一标准进行比较。

3.3.1 实验对比结果

通过 3.2 节的实验，结合之前提出的评估指标，实验结果可归纳为表 3-2。其中 TPS 与共识确认时间可体现出共识算法的性能，为确保共识确认时间准确，本文列举了重要应用的数据。PBFT 与 Raft 的性能可以满足大部分需求，而 PoW 在处理大量交易的场景下无法立即确认交易。

参与共识过程的节点数量体现了共识算法的去中心化程度，也能间接保证共

识过程的公平性。PBFT 与 Raft 在共识节点数量大于 30 后性能开始显著下降，适用于联盟链或私有链。PoW 则是全部节点参与共识过程，适用于公有链。

拜占庭容错能力上，PoW 最好，可以容忍总节点数 $1/2$ 的恶意节点^[2]，PBFT 可容忍总节点数 $1/3$ 的恶意节点^[18]，Raft 不具有拜占庭容错能力^[11]，故 Raft 无法应用在可能有恶意节点的场景中。

资源消耗作为重要的指标，PoW 的计算难题所带来的过高的电力消耗不适宜作为区块链长期发展，而 PBFT 与 Raft 并不需要节点进行大量运算，主要在于消息通信的资源消耗，故相比 PoW 足够节能。

表 3-2 经典共识算法对比

共识算法 评估指标	PBFT	Raft	PoW
TPS	211	188	7
共识确认时间	Fabric: 1 秒 FISCO-BCOS: 1 秒	FISCO-BCOS: 1 秒	比特币: 60 分钟 以太坊: 1 分钟
共识节点数量	小于 30	小于 30	全部节点
拜占庭容错能力	小于 $1/3$ 总节点数	无	小于 $1/2$ 总节点数
资源消耗	低	低	高

PBFT、PoW、Raft 分别代表了前文归纳的三类共识算法：确定性拜占庭容错共识算法、概率性拜占庭容错共识算法、崩溃容错共识算法。经过实验与分析，现总结这些共识算法存在的问题如表 3-3。

表 3-3 经典共识算法缺陷

算法	缺陷
PBFT	节点数量固定，共识节点数大于 30 后性能下降，C/S 请求模式不符合去中心化
Raft	无拜占庭容错能力，共识节点数大于 30 后性能下降，过于依赖主节点
PoW	性能低下，计算资源浪费且消耗大量电力

3.3.2 改进目标

经过以上实验归纳，本文总结出改进目标，即改进后的共识算法需具有高性能、高扩展性以及安全性。

3.3.2.1 高性能

对于区块链系统，性能主要体现在 TPS 以及共识确认时间，对于一般用户而言 TPS 与共识确认时间是可感知的，如果性能较差会影响其使用体验。对于商业应用，TPS 是最重要的，可以直接反映出整个系统处理事务的能力，高 TPS 是吸引企业使用的一大因素。

3.3.2.2 高扩展性

扩展性指的是区块链可以动态添加或减少节点，且在节点增加时仍能保证高性能。

区块链的一大特征是去中心化，如果共识节点只有固定的一些节点能够参与，

便丧失了去中心化特征，整个系统的公平性便难以保证。

另外，共识算法在增加参与共识的节点数后性能可能会明显下降，如 PBFT，在需要参与共识节点数较多时，区块链将失去可用性。所以增加节点后的性能表现也是扩展性所关注的。

3.3.2.3 高安全性

拜占庭容错能力、防止双花攻击、防止女巫攻击、保护打包节点等构成了区块链系统的安全性。

拜占庭容错包括了容忍节点的机器故障以及节点的恶意攻击。对于不同的共识算法，容忍恶意节点的数量是不同的。

双花攻击是区块链中常见的攻击，攻击者在掌握足够强大的算力后可以改写既定区块，使得已经消费的代币重新取回，这对于区块链的经济生态是致命的。女巫攻击，即攻击者可以根据共识算法的机制伪造大量节点影响其共识结果。由于共识算法通常需要选出打包节点，而部分共识算法的选主逻辑较为简单，攻击者很容易提前计算出打包节点并攻击它从而影响正常的共识过程。

总而言之，区块链中安全性主要体现在拜占庭容错能力以及抵御其它区块链中的常见攻击的能力，如果安全性较差，区块链系统将会崩溃。

第四章 RBFT 算法设计与实现

本章提出一种共识算法，该算法结合了 PoS、PBFT 等经典共识算法，通过 PoS 作为依据决定共识节点集，VRF 值用于选举得出每一轮参与共识流程的节点，改进后的 PBFT 进行共识。该共识算法为随机选取主节点的拜占庭容错共识算法，简称 RBFT。本章详细介绍了 RBFT 共识算法的改进思路，并在 Quorum 中得到实现。

4.1 算法改进

经过第三章的详细分析，本节提出主要改进思路，以得出符合改进方向的共识算法。

通过上文的介绍分析，现总结 PBFT 适宜在联盟链使用的优点如下：

- 1) 节点数量小于 30 时性能极佳；
- 2) 拥有拜占庭容错能力；
- 3) 不会产生分叉；
- 4) 达成一致性的资源消耗很少；
- 5) 可以脱离代币的方式进行共识，这对于不需要代币的场景很有价值。

综上优点，PBFT 适合在联盟链应用。然而 PBFT 作为传统的分布式一致性算法，它的节点数量是固定的，且其消息通讯基于 C/S 请求响应模式。考虑到区块链中节点进出频繁且各节点之间应相互平等，这两个缺陷对区块链系统而言往往无法承受，故需对 PBFT 进行两点改进：

- 1) 节点增减动态化，即区块链系统需能动态感知共识节点的增加与减少；
- 2) 由 C/S 模型改为 P2P 网络模型。

4.1.1 选取共识节点的改进

选取共识节点主要通过 PoS 与 VRF 加以改进。下面首先介绍 PoS 带来的改进，随后介绍 VRF 的引入。

PBFT 无法防范由 Douceur 提出的女巫攻击，即攻击者可以伪造大量节点影响共识结果^[38]。而 PoS 极大地提高了攻击者伪造节点的成本从而杜绝女巫攻击，保证共识过程的正确。此外，PoS 是一种可以解决经济博弈的共识算法，能更大的保证公平性。如图 4-1 所示，PoS 提供的权益证明作为一种公平的方式可以很好地从普通节点集中选出共识节点集。另外，PoS 可以避免大量算力造成的资源消耗，两者的结合对于联盟链而言可以做到高效、安全、公平、节能。PoS 详细的应用将在设计部分进行介绍。

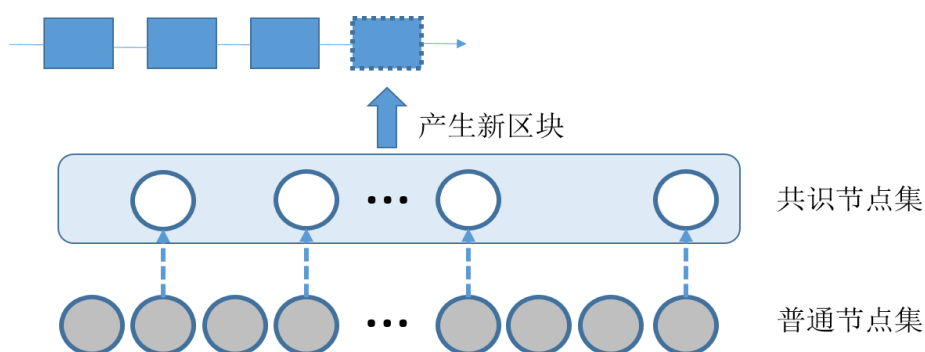


图 4-1 PoS 得到共识节点集

PBFT 因其 $O(n^2)$ 的消息广播时间复杂度导致性能在节点数量足够多时大幅下滑，而无论在公有链、联盟链或是私有链中，都可能有较多节点参与共识，如果只是固定的少数节点参与共识，便失去了区块链的去中心化带来的公平性，安全性也会随之降低，因为固定的少数共识节点被攻击后区块链系统就会瘫痪。因此，研究 PBFT 在有较多节点参与共识时性能依旧保持出色是很有必要的。

可验证随机函数（Verifiable Random Function, VRF）由 Micali 等提出，只有 VRF 私钥的持有者才能计算哈希，但是具有相应公钥的任何人都可以验证哈希的正确性^[39]。VRF 的关键应用是为存储在基于哈希的数据结构中的数据提供针对离线枚举（例如字典攻击）的隐私性。在此应用中，Prover 保有 VRF 私钥并使用 VRF 哈希在输入数据上构建基于哈希的数据结构。由于 VRF 的性质，只有 Prover 才能给出有关哈希正确性的证明。知道其 VRF 公钥的任何人都可以验证 Prover 的证明是否正确，却不能对存储在数据结构中的数据做出推断。

VRF 附带一个密钥生成算法，可以生成公钥 PK 和私钥 SK。Prover 通过其私钥对某个输入 α 进行哈希处理可得到随机输出 β ，如公式 4.1 所示。

$$\beta = \text{VRF_hash}(\text{SK}, \alpha) \quad (4.1)$$

Prover 还使用私钥来构造零知识证明 π ，可用来证明 β 是正确输出，如公式 4.2 所示。

$$\pi = \text{VRF_prove}(\text{SK}, \alpha) \quad (4.2)$$

任何人都可直接通过 π 获得 β ，如公式 4.3 所示。

$$\beta = \text{VRF_proof_to_hash}(\pi) \quad (4.3)$$

也就有：

$$\text{VRF_hash}(\text{SK}, \alpha) = \text{VRF_proof_to_hash}(\text{VRF_prove}(\text{SK}, \alpha)) \quad (4.4)$$

故可用后者作为前者的表示。 π 允许 Verifier 持有公钥 PK 以验证 β 是否是基于 PK 下的 α 的正确输出 β 。因此，VRF 还附带了算法：

$$\text{VRF_verify}(\text{PK}, \alpha, \pi) \quad (4.5)$$

这个算法首先计算 $\beta = \text{VRF_proof_to_hash}(\pi)$ ，若计算出的 β 等于 PK 下的正确输出 β ，输出 VALID，否则输出 INVALID。

VRF 具有以下性质：

- 1) 唯一性：对于任何固定的 PK 和任何输入 α ，都有一个唯一的 VRF 输出 β ，并且 β 是可以被验证的。
- 2) 抗碰撞性：给定一个消息 m_0 ，很难找到另一个消息 m_1 使得 $\text{Hash}(m_0) = \text{Hash}(m_1)$ 。
- 3) 伪随机性：伪随机性确保当攻击者在没有相应的 VRF 证明 π 的情况下看到 VRF 哈希输出 β 时， β 看起来就是随机值。

VRF 可做到无交互抽签，各节点只需要在本地通过私钥 SK 计算 β 用以抽签并计算 π ，将符合条件的 β 与 π 广播出去后其它节点利用公钥 PK 与 π 验证 β 的正确性即可。所有共识节点维护一张 PoS 表，通过取 β 中的某一位，随机处理后将其作为 PoS 表的下标，即可随机选出节点。通过这种方式，每次计算出多个节点成为提案节点、验证节点、确认节点，提升了整个过程的容错性与公平性。由于一个区块的 β 是可验证的，在不分叉的情况下所有节点对于同一区块的 VRF 就是一致的。在 PoS 表中选择节点设计为顺序进行，则每个 VRF 值代表了一个提案节点的顺序，这个顺序也是共识一致的。引入 VRF 后的共识过程如图 4-2 所示。引入 VRF 后，PBFT 在共识节点数超过 30 的场景可以通过 VRF 的无交互抽签得出每一轮最终进行共识的提案节点集、验证节点集、确认节点集，并且该结果是随机的、可验证的，能够给 PBFT 带来更好的扩展性以及安全性，理由如下：

- 1) VRF 可以控制抽中的节点个数，从而精准缩小共识节点集，使得实际进行 PBFT 三阶段过程的节点不超过阈值，保证 PBFT 在多节点场景性能仍然优秀。
- 2) VRF 改善了 PBFT 的选主方式，PBFT 的取余法易暴露主节点的顺序从而被 DoS 攻击，影响主节点打包区块。引入 VRF 后，由于各个节点在本地抽签，攻击者无法提前得知主节点。这提升了打包过程的安全性。
- 3) 每轮共识过程中，提案节点、验证节点、确认节点都是由 VRF 随机选出的，攻击者难以预测，从而极大增强了共识算法整体的抗攻击性。
- 4) VRF 产生的结果是可验证的，任何节点都能得到中签节点广播的零知识证明 π ，对 π 验证可以得出 VRF 结果是否正确，因此，VRF 的结果是无法伪造的。

此外，VRF 与 PoS 的结合还能带来公平性的提升，例如 PoS 的权益可以作为抽签的概率来源，权益越高的节点可以得到更高的中签概率。

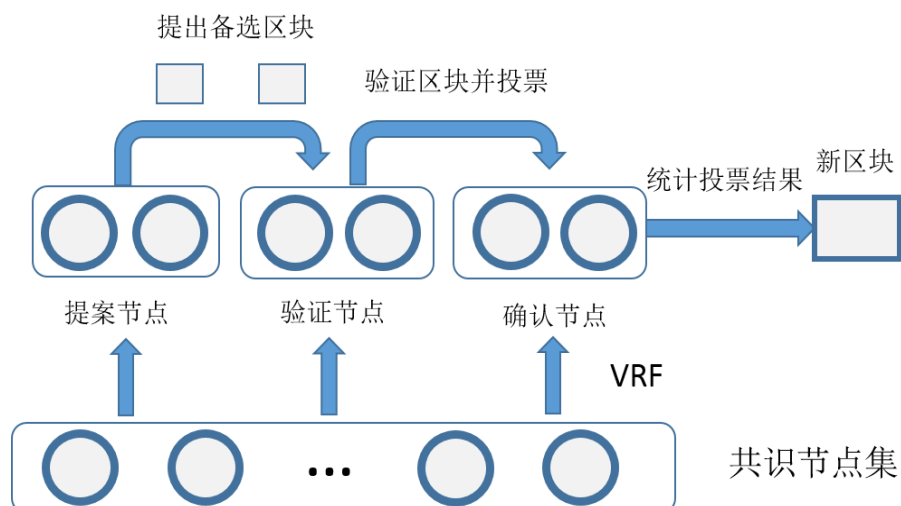


图 4-2 引入 VRF 的共识过程

4.1.2 PBFT 共识流程的改进

除上述改进之外，RBFT 还对 PBFT 的共识流程细节提出多方面的改进：

- 1) 使用 Gossip 网络建立节点之间的连接。一般的 P2P 网络需要所有节点彼此直接连接，在区块链环境中难以实现。而 Gossip 允许节点间的弱连接，即节点既可以直接连接，也可以通过其它节点进行中继，消息同样可以正常广播确保共识结果。
- 2) 允许节点在异步网络环境中，可以接收无法在当前状态下处理的消息。例如，节点可以在 NEW ROUND 上接收 COMMIT 消息。当验证程序收到将来的消息时，它会将消息暂存，并尽可能在稍后尝试处理。
- 3) 为了加速共识过程，如果节点在接收到 $2F + 1$ 个 PREFARE 消息之前接收了 $2F + 1$ 个 COMMIT 消息，节点将直接进入到了 COMMITTED 状态，这样就不必一直等待 PREPARE 消息。

4.2 算法实现

本节设计了实现所需的算法流程，随后介绍了关键模块的实现。

4.2.1 算法流程设计

本小节介绍了 RBFT 核心算法的设计思路，并给出了伪代码，包括 VRF 的抽签算法以及共识主要流程。

4.2.1.1 共识流程

VRF 抽签算法在 RBFT 中随机选出最终参与共识的三类节点并提供中签的证明。定义抽签算法的输入：SK：用户私钥， α ：伪随机种子， τ ：成员中签数量， w ：用户权重， W ：总权重，type：成员类型。算法流程如下：

- 1) 算法首先使用公式 4.1 与公式 4.2 分别计算出随机哈希值 β 与零知识证明 π ，其中哈希值由私钥和输入参数决定，他人不可伪造；证明 π 可以让任

何知晓该用户公钥 PK 的用户验证 β 确实由该输入推导出来, 令 β 的长度为 βlen , β 的随机范围在 $[0, 2^{\beta\text{len}} - 1]$ 。

- 2) 为了能够使用户被选中的概率和其在 PoS 中所拥有的权益相对应, RBFT 将用户 User 按照其拥有多少单位的代币分割成 sub-users, 也即: 假设用户 i 拥有 w_i 数量的代币, 那么该用户就拥有 w_i 个 sub-users。给出 User (i, j), 其中 $j \in \{1, \dots, w_i\}$ 代表了 i 拥有的第 j 个代币, 每一个单位代币被选中的概率 $p = \frac{\tau}{w}$ 。

- 3) 用户的 w 个 sub-users 中被选中 k 个的概率遵循如下二项分布:

$$B(k; w, p) = \binom{w}{k} p^k (1-p)^{w-k} \quad (4.6)$$

其中 $\sum_{k=0}^w B(k; w, p) = 1$ 。

- 4) 为了确定用户 w 个 sub-users 中多少个 sub-user 被选中, 抽签算法将 $[0, 1)$ 区间分割成连续的区间 I^j , 区间计算如下:

$$I^j = \left[\sum_{k=0}^j B(k; w, p), \sum_{k=0}^{j+1} B(k; w, p) \right) \quad (4.7)$$

其中 $j \in \{1, \dots, w\}$ 。

- 5) 如果 $\frac{\beta}{\beta\text{len}}$ 落在区间 I^j , 那么该用户共有 j 个 sub-users 被选中, 该数字 j 能

够通过 被其他用户验证, j 大于 0 就证明本节点在本轮次中被选中为共识节点。

抽签算法提供两个关键能力。一是随机的哈希值 β 均匀分布于 0 和 $2^{\beta\text{len}} - 1$ 之间, 节点权益占比越高, 抽中的概率越大。二是攻击者在不知道节点的 SK 情况下, 无法根据权益猜测哪些节点被抽中了, 保证了共识节点的安全性。

验证抽签结果的算法用到公式 4.5, 所有节点能够根据中签节点的 PK 及其提供的零知识证明 π 、随机种子 α 来验证抽签结果是否正确。验证抽签算法将得到中签节点的 sub-user 数量, 未被选中则返回 0。

为了保证抽签算法的正确运行, 在相同轮次中, 随机种子 α 对于每个共识节点应是一致的, 本文的随机种子 α 由上一个区块的易变字段进行计算得出。随机种子相应算法如下:

- 随机种子的初始值产生于创世区块, 由参与第一轮共识的共识节点通过随机算法产生。
- 在共识算法运行的某轮次 r 中, 一旦新区块产生, α 会在提取所需字段值计算后得到更新的 α' 。假设提案节点为 p , 其私钥为 SK_u , 由 $VRF_{SK_u}(\alpha||r)$ 给出 $\langle \alpha', \pi \rangle$, p 在新区块产生时会向全网广播 α' , 同时 α' 以及 π 写入新区块。
- 抽签算法的随机种子会每隔一定轮次更新一次防止攻击。

RBFT 是一种状态机复制算法，每个共识节点都维护一个状态机副本以达到一致性。假设被 VRF 选出的共识节点总数为 $3F+1$ ，状态及状态转移动作如表 4-1。

表 4-1 状态定义

状态	动作
NEW ROUND	提案节点发送新的区块提案，验证节点等待 PRE-PREPARE 消息
PRE-PREPARED	验证节点已收到 PRE-PREPARE 消息并广播 PREPARE 消息，然后它等待 $2F+1$ 个 PREPARE 或 COMMIT 消息
PREPARED	验证节点已收到 $2F+1$ 个 PREPARE 消息并广播 COMMIT 消息，然后它等待 $2F+1$ 个 COMMIT 消息
COMMITTED	确认节点已收到 $2F+1$ 个 COMMIT 消息，将提案的区块插入区块链
FINAL COMMITTED	成功地将新块插入到区块链中，准备进行下一轮
ROUND CHANGE	验证节点与确认节点等待 $2F+1$ 个 ROUND CHANGE 消息

RBFT 的状态转移图如图 4-3 所示。由新轮次到预准备状态时，被选中的提案节点首先从交易池收集当前所有交易，广播新区块提案后提案节点进入预准备状态。所有被选中的验证节点将进入预准备阶段并对收到的预准备消息进行校验，通过 VRF 证明对提案节点进行身份验证。通过验证后验证节点将广播准备消息，验证节点收到足够多的准备消息后将广播提交消息，被选中的确认节点收到足够多个提交消息后将把提案区块插入区块链，等待进入下一轮。

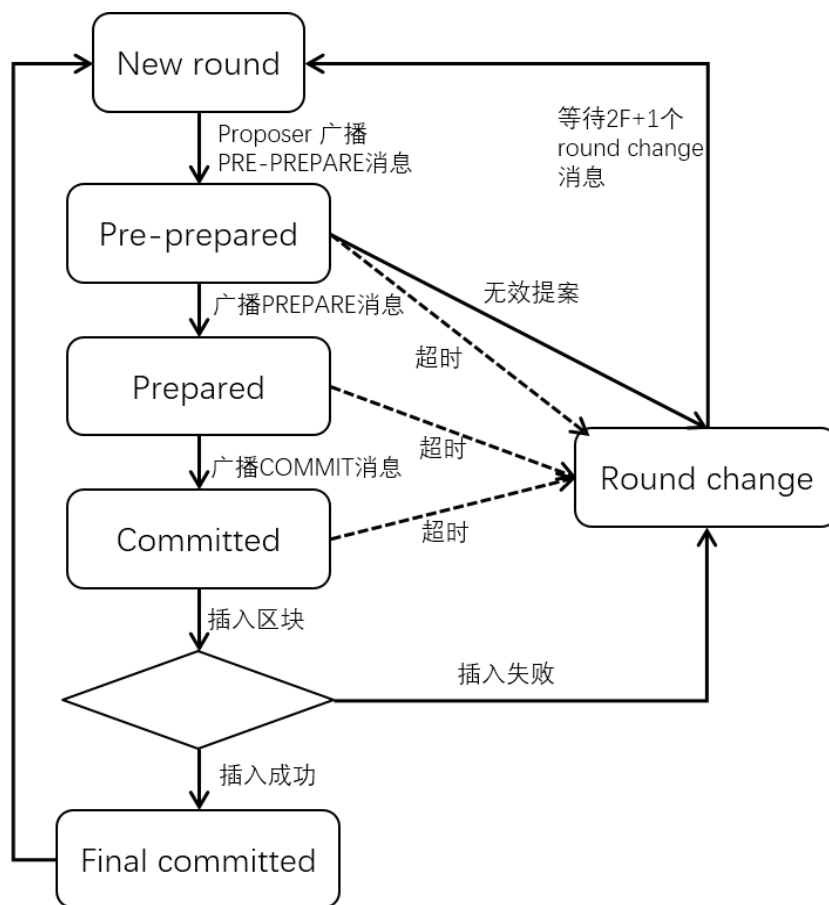


图 4-3 RBFT 状态转移图

详细算法流程如下：

- 1) 客户端通过 P2P 网络广播交易数据请求, 并附上交易发起者的签名;
- 2) PoS 确定共识节点集;
- 3) 网络中所有参与共识节点在 P2P 网络中独立监听全网交易, 并缓存在本地内存中;
- 4) 提议新的区块:
 - a) 节点对当前区块高度和上一区块中的 VRF 值做签名, 然后计算哈希, 根据哈希值独立判断自身能否成为当前高度区块的潜在 proposer;
 - b) 如果节点判断自己为当前区块高度的 proposer:
 - i. 打包当前以及收集到的未共识的交易请求, 构建新的区块;
 - ii. 并对此区块数据和区块哈希进行签名, 将区块和签名向 P2P 网络中广播;
- 5) 所有节点将等待 T_{block} 时间, 独立监听网络中的区块 proposal, 并将收到的区块 proposal 缓存在本地内存中;
- 6) 新区块的分级共识:
 - a) T_{block} 后, 网络中节点独立判断自身是否为当前高度各个区块 proposal 的验证节点;
 - b) 如果节点判断自己为当前区块 proposal 的验证节点:
 - i. 验证所有区块 proposal 的正确性和完整性;
 - ii. 验证所有区块 proposal 中的 VRF 值签名的正确性;
 - iii. 验证区块 proposer 的身份的合法性;
 - iv. 对所有 proposal 的 VRF 值签名计算哈希, 选取哈希最小者的区块 proposal 作为当前节点所选择的真正区块;
 - v. 对所选择的区块 proposal 的哈希进行签名, 将区块哈希和签名向 P2P 网络中广播;
 - c) T_{hash} 后, 执行节点更新算法阶段, 独立判断自身是否应该在此步骤中参与对区块 proposal 的验证;
 - d) 如果节点判断自己在当前阶段为区块 proposal 的验证节点:
 - i. 同样执行上述的区块 proposal 验证工作;
 - ii. 同样对所选 block proposal 的哈希进行签名, 将区块哈希和签名向 P2P 网络中广播;
- 7) 网络中所有节点都将在 P2P 网络中独立监听区块验证节点所广播的共识消息:
 - a) 验证共识消息的有效性和完整性;
 - b) 验证发出共识消息的节点身份的合法性;
 - c) 将验证消息缓存在本地内存中;
- 8) 对新区块的拜占庭投票:

- a) T_{hash} 后, 执行节点更新算法阶段, 独立判断自身是否在此步骤中参与对新区块的投票;
- b) 如果节点判断自己应当参与此轮投票:
 - i. 计算不同 block proposal 所收到的共识验证的票数;
 - ii. 对共识的结果进行投票;
 - iii. 对投票结果进行签名, 并将所投票的区块哈希, 投票结果及其签名向 P2P 网络中广播;
- c) 网络中的所有节点将按照区块链的配置, 对区块的投票结果再进行多轮投票, 每轮投票都将在 T_{hash} 后, 网络中所有节点独立完成;
- 9) 网络中所有节点也在 P2P 网络中持续监听广播的投票消息, 并对投票消息进行有效验证;
- 10) 对新区块的最终签名:
 - a) 完成最后一轮拜占庭投票后, 执行节点更新算法阶段, 判断自身是否参与对新区块的最终签名;
 - b) 如果节点判断自己应当参与区块签名:
 - i. 根据收集到的投票结果, 判断所投票的 block proposal 是否共识;
 - ii. 如果判断没有达到 $2F+1$ 票数, 将此轮新的区块共识结果设置为空块;
 - iii. 将自身做出判断的投票消息进行打包, 并签名;
 - iv. 将最终的区块哈希和自身对结果的签名向 P2P 网络中广播;
- 11) 网络中所有节点在 P2P 网络中持续监听区块最终签名消息, 并对区块签名消息进行有效验证。
 - a) T_{hash} 后, 将根据所收到的区块签名消息, 得到当前高度的区块哈希;
 - b) 根据区块哈希, 和之前在网络中收集到的区块 proposal 消息相比较, 得到当前高度共识的最终区块。

4.2.1.2 核心伪代码

本节通过伪代码介绍了关键算法的实现逻辑, 6 个算法都是 RBFT 中核心过程的体现。

首先是加密抽签算法, 在本机通过输入本节点的私钥以及所有节点都能得到的其它参数, 经过计算即可得到 VRF 随机哈希值与中签结果。

算法 1 加密抽签算法

输入: 节点私钥 SK , 随机种子 α , 成员类型 $type$, 成员中签数量 τ , 用户权重 w , 总权重 W

输出: 随机哈希值 β , 零知识证明 π , 被选中的 sub-users 数量 j

1. **function** Sortition($SK, \alpha, type, \tau, w, W$)

```

2.    $\beta \leftarrow \text{VRF\_hash}(\alpha || \text{type})$ 
3.    $\pi \leftarrow \text{VRF\_prove}(\alpha || \text{type})$ 
4.    $p \leftarrow \frac{\tau}{W}$ 
5.    $j \leftarrow 0$ 
6.   while  $\frac{\pi}{2^{\beta \text{len}}} \notin (\sum_{k=0}^j B(k; w, p), \sum_{k=0}^{j+1} B(k; w, p))$  do
7.        $j++$ 
8.   end while
9.   return  $(\beta, \pi, j)$ 
10. end function

```

验证抽签算法是其它节点对中签节点进行验证时的算法, 某节点收到中签节点的消息后, 中签节点的 PK 与 π 是公开的, 通过公式 4.5 即可初步判断是否合法, 之后再用抽签算法的思路进行验证, 如果通过验证将返回中签子用户数量。

算法 2 验证抽签算法

输入: 节点公钥 PK , 随机哈希值 β , 零知识证明 π , 随机种子 α , 成员类型 type , 成员中签数量 τ , 用户权重 w , 总权重 W

输出: 被选中的 sub-users 数量 j

```

1. function VerifySortition( $PK, \beta, \pi, \alpha, \text{type}, \tau, w, W$ )
2.   if  $\neg \text{VRF\_verify}(PK, \alpha, \pi || \text{type})$  then
3.       return 0
4.   end if
5.    $p \leftarrow \frac{\tau}{W}$ 
6.    $j \leftarrow 0$ 
7.   while  $\frac{\pi}{2^{\beta \text{len}}} \notin (\sum_{k=0}^j B(k; w, p), \sum_{k=0}^{j+1} B(k; w, p))$  do
8.        $j++$ 
9.   end while
10.  return  $j$ 
11. end function

```

RBFT 共识算法用以得到共识区块, 经过一系列验证及投票, 确保新区块的正常产生。该算法用到了其它所有算法。

算法 3 RBFT 共识算法

输入: 合约状态 state , 轮数 round , 提案区块 block

输出: 共识区块 block2

```

1. function RBFT( $\text{state}, \text{round}, \text{block}$ )
2.   ▷ 投出区块哈希
3.   MemberVote( $\text{state}, \text{round}, \text{RBFT\_ONE}, \tau, \text{block}$ )
4.    $\text{block1} \leftarrow \text{CountVotes}(\text{state}, \text{round}, \text{RBFT\_ONE}, T_{\text{STEP}}, \tau, \lambda_{\text{BLOCK}} + \lambda_{\text{STEP}})$ 

```

```

5.      ▷返回为空或超时则再次发起投票
6.       $empty\_hash \leftarrow H(\text{Empty}(\text{round}, H(\text{state.last\_block})))$ 
7.      if  $block1 = \text{TIMEOUT}$  then
8.          MemberVote ( $\text{state}, \text{round}, \text{RBFT\_TWO}, T_{STEP}, empty\_hash$ )
9.      else
10.         MemberVote ( $\text{state}, \text{round}, \text{RBFT\_TWO}, T_{STEP}, block1$ )
11.      end if
12.       $block2 \leftarrow \text{CountVotes}(\text{state}, \text{round}, \text{RBFT\_TWO}, T_{STEP}, \tau, \lambda_{STEP})$ 
13.      if  $block2 = \text{TIMEOUT}$  then
14.          return  $empty\_hash$ 
15.      else
16.          return  $block2$ 
17.      end if
18. end function

```

成员投票算法可以验证该节点是否在当前步骤当选为共识节点，首先检查自身是否是合法的共识成员，若是则广播一条消息，消息包含了该节点的公钥、某提案区块哈希以及零知识证明等供其它节点验证。

算法 4 成员投票算法

输入： 合约状态 $state$ ，轮数 $round$ ，共识阶段 $step$ ，成员中签数量 τ ，区块哈希 $value$

```

1. function MemberVote( $state, round, step, \tau, value$ )
2.     ▷用算法 1 检查该节点是否是共识成员
3.      $type \leftarrow (\text{"committee"}, round, step)$ 
4.      $(sorthash, \pi, j) \leftarrow \text{Sortition}(\text{user.SK}, \text{state}.\alpha, \tau, type, \text{state.weight}[\text{user.PK}], \text{state.W})$ 
5.     ▷如果是共识成员，则投出自己支持的区块
6.     if  $j > 0$  then
7.         Gossip( $\text{user.PK}, \text{Signed}_{\text{user.SK}}(\text{round}, step, sorthash, \pi, H(\text{state.last\_block}), value)$ )
8.     end if
9. end function

```

统计票数算法首先从消息队列中收集当前轮次和步骤的消息。它通过为每条消息调用算法 6 来处理投票并确保投票有效。当某个区块哈希 $value$ 超过 $T \cdot \tau$ 票，算法返回值即为该 $value$ 。 T 是预期共识节点规模（ T 大于 $2/3$ ），可以确定 RBFT 的投票门槛。如果在设定的投票超时时间 λ 内没有收到足够的消息，则返回超时标记 TIMEOUT。

算法 5 统计票数算法

输入： 合约状态 $state$ ，轮数 $round$ ，共识阶段 $step$ ，预设比例 T ，成员中签数量

τ , 投票超时时间 λ

输出: 区块哈希 *value*

```

1. function CountVotes(state, round, step, T,  $\tau$ ,  $\lambda$ )
2.   start  $\leftarrow$  Time()
3.   counts  $\leftarrow$  {}
4.   voters  $\leftarrow$  {}
5.   msgs  $\leftarrow$  incomingMsgs[round, step].iterator()
6.   while True do
7.     m  $\leftarrow$  msgs.next()
8.     if m = nil then
9.       if Time() > start +  $\lambda$  then return TIMEOUT
10.      end if
11.    else
12.      (votes, value, sorthash)  $\leftarrow$  ProcessMsg(state,  $\tau$ , m)
13.      if PK  $\in$  voters or votes = 0 then continue
14.      end if
15.      voters  $\cup$  = {PK}
16.      counts[value] += votes
17.      if counts[value] > T ·  $\tau$  then
18.        return value
19.      end if
20.    end if
21.  end while
22. end function

```

消息处理算法不仅返回消息中包含的区块哈希 *value*, 还返回与该区块相关的投票数。如果消息不是来自选定的共识成员则算法返回 0 表示没有投票。如果共识成员被抽中多次, 返回的投票数也会反映出来。

算法 6 消息处理算法

输入: 合约状态 *state*, 成员中签数量 τ , 消息 *m*

输出: 区块票数 *votes*, 区块哈希 *value*

```

1. function ProcessMsg(state,  $\tau$ , m)
2.   (PK, signed_m)  $\leftarrow$  m
3.   if VerifySignature(PK, signed_m)  $\neq$  OK then
4.     return (0, nil, nil)
5.   end if
6.   (round, step, sorthash,  $\pi$ , hprev, value)  $\leftarrow$  signed_m
7.   if hprev, H(state.last_block) then
8.     return (0, nil, nil)
9.   end if

```

10. $votes \leftarrow \text{VerifySortition}(PK, \text{sorthash}, \pi, \text{state}.\alpha, \tau, (\text{"committee"}, \text{round}, \text{step}), \text{state.weight}[PK], \text{state.W})$
11. **return** ($votes, value$)
12. **end function**

4.2.2 关键模块实现

本小节介绍了 RBFT 的具体实现,给出了关键模块的实现思路与代码。RBFT 系统架构如图 4-4 所示。

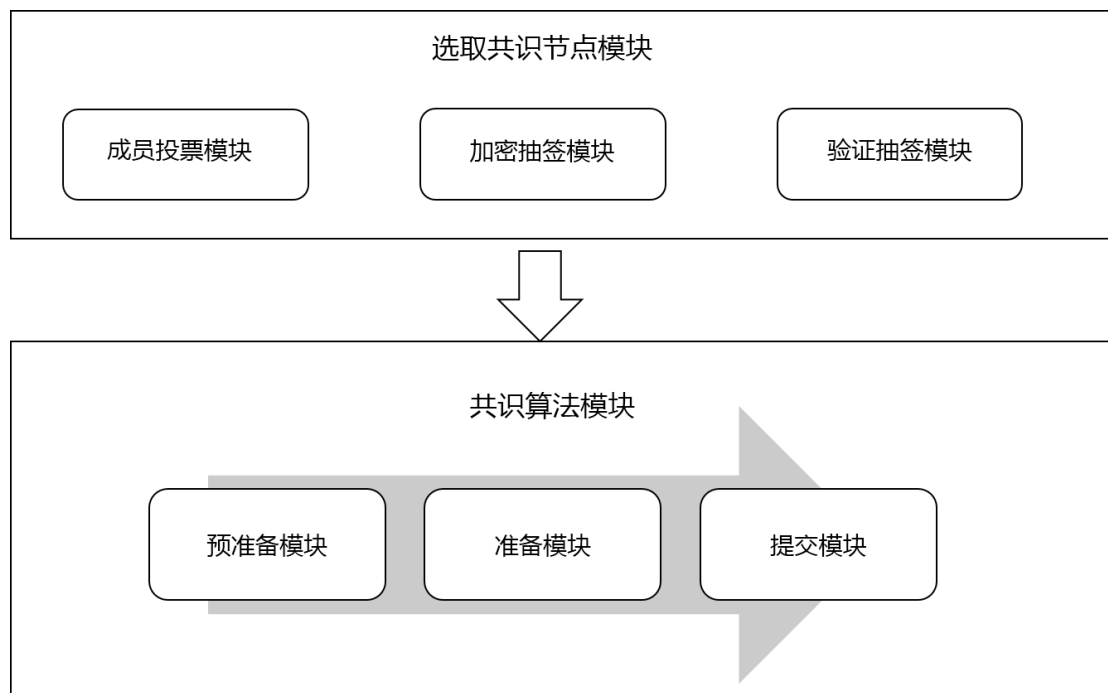


图 4-4 RBFT 系统架构图

本文使用 Go 语言实现 RBFT。区块链需要高性能,业界往往会使用 Go 语言作为区块链项目的开发语言。Go 相比 Java 的优势体现在高效以及代码量更小,相比 C++ 的优势在于减少了开发难度。详细的实现环境如表 4-2 所示。

表 4-2 实现环境

环境名称	说明
开发系统	Windows 7
运行环境	Windows 或 Ubuntu
内存	16G
编译工具	Go 1.10.7, IDE 为 Goland

由于 Quorum 已经成功应用于联盟链,本文在 Quorum 基础上实现 RBFT 共识算法,以期在企业中进行实际应用。本节将从选取共识节点模块以及三阶段流程模块进行介绍。

4.2.2.1 选取共识节点模块

选取共识节点模块主要包含 VRF 以及 PoS 的加入,下文将介绍其实现。

VRF 的有关信息需要记录在区块中，其结构体设计如表 4-3。

表 4-3 VRF 存储结构体

字段名	类型	字段含义
Proposer	[20]byte	本轮提案节点
VrfValue	[]byte	本轮 VRF 的随机哈希值 β
VrfProof	[]byte	本轮提案者提供的零知识证明 π
LastBlockNum	uint64	上一区块号

VRF 随机种子的结构体字段如表 4-4。

表 4-4 VRF 随机种子结构体

字段名	类型	字段含义
BlockNum	uint64	当前区块号
PrevProposer	[20]byte	上一区块的提案者的账户地址
BlockRoot	[32]byte	区块根的哈希值
VrfValue	[]byte	VRF 的随机哈希值 β

广播区块提案的同时，需要在预准备阶段广播 VRF 相关信息，采用 RLP 进行序列化并与其它消息字段统一格式。实现后运行情况如图 4-5。

图 4-5 RBFT 计算 VRF 值运行情况

本文给出选取共识节点的部分关键代码：

```
func (self *BackEnd) BuildParticipantConfig(valSet istanbul.ValidatorSet, blkNum uint64,
block *types.Block) istanbul.ValidatorSet {
    vrfValue := GetParticipantSelectionSeed(block)
    cfg := &BlockParticipantConfig{
        BlockNum: blkNum,
        Vrf:      vrfValue,
    }
    s := 0
    cfg.Proposers = calcParticipantPeers(valSet, cfg, s, s+types.MAX_PROPOSER_COUNT)
    var newValSet istanbul.ValidatorSet
    b := []byte{}
    for i := 0; i < len(cfg.Proposers); i++ {
        b = append(valSet.GetByIndex(uint64(cfg.Proposers[i])).Address().Bytes())
    }
}
```

```

    newValSet = validator.NewSet(validator.ExtractValidators(b), istanbul.RoundRobin)
    return newValSet
}
func calcParticipantPeers(valSet istanbul.ValidatorSet, cfg *BlockParticipantConfig, start,
end int) []uint32 {
    peers := make([]uint32, 0)
    peerMap := make(map[uint32]bool)
    var cnt uint32
    for i := start; i < end; i++ {
        peerId := calcParticipant(len(valSet.List()), cfg.Vrf, uint32(i))
        if peerId == math.MaxUint32 {
            return []uint32{}
        }
        if _, present := peerMap[peerId]; !present {
            peers = append(peers, peerId)
            peerMap[peerId] = true
            cnt++
            if cnt >= 4 {
                return peers
            }
        }
    }
    return peers
}

```

PoS 在 RBFT 中通过切片（slice）的方式实现。切片是 Go 语言中可以动态分配内存的数据结构，可以理解为动态的数组。因为 PoS 的节点表很可能新增或删除节点，所以采用切片实现。PoS 表在创世块中进行初始化，首先得到当前所有节点的 stake 值并排序，随后将节点依序插入 PoS 切片中即完成初始化。PoS 表将作为链的配置与节点、超时时间、视图编号等参数一同写入配置项。此外还有一个 TotalPoS 字段作为所有节点的总权益值。

当节点的权益值变化时，首先需要经过一系列校验，之后更新该节点的权益值并把该节点保存至节点池。还需更新 TotalPoS，随后通过调用智能合约进行更新，写入配置。选取了 PoS 部分的关键代码如下：

```

MainnetChainConfig = &ChainConfig{
    RBFT: &RBFTConfig{
        N:          7,
        C:          2,
        K:          7,
        L:          112,
        MinInitStake: 100000,
        VrfValue:
        "1c9810aa9822e511d5804a9c4db9dd08497c31087b0daafa34d768a3253441fa20515e2f30f8174
        1102af0ca3cefc4818fef16adb825fbba8cad78647f3afb590e",
    },
}

```

VrfProof:

"c57741f934042cb8d8b087b44b161db56fc3ffd4ffb675d36cd09f83935be853d8729f3f5298d12d6fd28d45dde515a4b9d7f67682d182ba5118abf451ff1988",

Peers: []*RBFTPeerStakeInfo{

{

Index: 1,

PeerPubkey:

"03348c8fe64e1defb408676b6e320038bd2e592c802e27c3d7e88e68270076c2f7",

Address: "AZavFr7sQ4em2NmQWdjLMY34tHMQzATWgx",

InitPos: 500,

},

},

},

}

4.2.2.2 RBFT 三阶段流程模块

RBFT 共识流程由预准备、准备、提交三阶段组成，下面将介绍三阶段实现。

预准备阶段对于 RBFT 尤为关键，因为该阶段节点需要对 VRF 的 VrfValue 与 VrfProof 进行计算并广播，节点在收到预准备消息时，要对这两个变量进行校验，从而确保节点选举的正确性。预准备阶段有关 vrf 的运行情况如图 4-6。

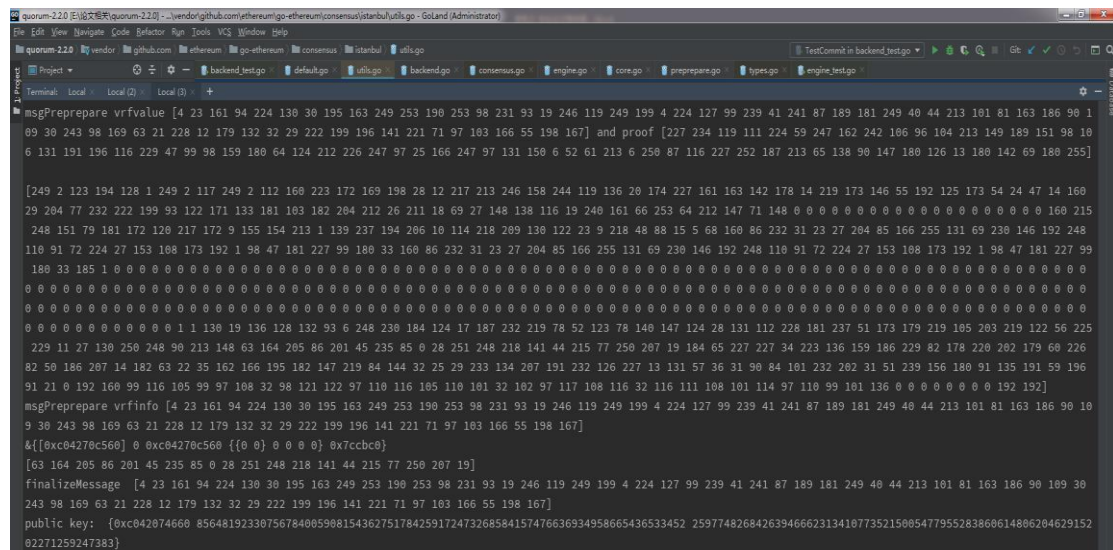


图 4-6 RBFT 广播消息截图

节选了 RBFT 预准备阶段发送预准备消息的核心方法代码如下，包含了 VRF 相关字段的计算并赋给消息进行序列化。

```
func (c *core) sendPreprepare(request *istanbul.Request) {
    if c.current.Sequence().Cmp(request.Proposal.Number()) == 0 && c.isProposer() {
        curView := c.currentView()
        preprepare, err := Encode(&istanbul.Preprepare{
            View:      curView,
            Proposal: request.Proposal,
        })
        preBlock := c.backend.GetBlock()
```



```

    prevvrf, err := preBlock.Info.VrfValue
    vrfValue, vrfProof, err := istanbul.ComputeVrf(c.backend.GetPrivateKey(),
preBlock.Number().Uint64()+1, prevvrf)
    msg := &message{
        Code:    msgPreprepare,
        Msg:     preprepare,
        Vrfvalue: vrfValue,
        Vrfproof: vrfProof,
        Proposer: c.backend.Address(),
        LastConfigBlockNum: c.backend.GetBlock().Number().Uint64(),
    }
    c.broadcast(msg)
}
}

```

RBFT 预准备阶段处理预准备消息的核心方法代码如下，重点在于对 PBFT 流程的改进，在该方法内决定是否跳过准备阶段直接进入提交阶段并投票。

```

func (c *core) handlePreprepare(msg *message, src istanbul.Validator) error {
    var preprepare *istanbul.Preprepare
    err := msg.Decode(&preprepare)
    if duration, err := c.backend.Verify(preprepare.Proposal); err != nil {
        if err == consensus.ErrFutureBlock {
            c.stopFuturePreprepareTimer()
            c.futurePreprepareTimer = time.AfterFunc(duration, func() {
                c.sendEvent(backlogEvent{
                    src: src,
                    msg: msg,
                })
            })
        } else {
            c.sendNextRoundChange()
        }
        return err
    }
    if c.state == StateAcceptRequest {
        if c.current.IsHashLocked() {
            if preprepare.Proposal.Hash() == c.current.GetLockedHash() {
                c.acceptPreprepare(preprepare)
                c.setState(StatePrepared)
                c.sendCommit()
            } else {
                c.sendNextRoundChange()
            }
        } else {
            c.acceptPreprepare(preprepare)
            c.setState(StatePreprepared)
        }
    }
}

```

```

        c.sendPrepare()
    }
}
return nil
}

```

选取 RBFT 准备阶段关键代码如下：

```

func (c *core) handlePrepare(msg *message, src istanbul.Validator) error {
    var prepare *istanbul.Subject
    err := msg.Decode(&prepare)
    c.acceptPrepare(msg, src)
    if ((c.current.IsHashLocked() && prepare.Digest == c.current.GetLockedHash()) ||
c.current.GetPrepareOrCommitSize() > 2*c.valSet.F()) &&
        c.state.Cmp(StatePrepared) < 0 {
        c.current.LockHash()
        c.setState(StatePrepared)
        c.sendCommit()
    }
    return nil
}

```

选取 RBFT 提交阶段关键代码如下：

```

func (c *core) handleCommit(msg *message, src istanbul.Validator) error {
    var commit *istanbul.Subject
    err := msg.Decode(&commit)
    c.acceptCommit(msg, src)
    if c.current.Commits.Size() > 2*c.valSet.F() && c.state.Cmp(StateCommitted) < 0 {
        c.current.LockHash()
        c.commit()
    }
    return nil
}

```

第五章 实验与评估

本章对 RBFT 算法进行实验与评估，首先介绍了实验环境，随后对 RBFT 进行了多项实验，最后对实验结果对比分析。

5.1 实验环境

本文采用 8 台服务器以完成所有实验，单机配置：处理器采用 Intel Xeon X3220 2.4GHz，内存大小 16GB，操作系统为 ubuntu18.04，网络带宽与 3.2 节实验时相同。

本文采用 Python 编写脚本进行实验，Python 版本为 3.6。在本机通过 web3 接口发送交易至服务器集群，得到区块链系统各项实验结果。

5.2 性能实验

本节主要对 RBFT 的性能进行实验，首先实验得出最佳环境，随后在多个节点测试得到 TPS 结果。另外对共识确认时间进行了实验。每次实验需要通过智能合约调用 RBFT 接口确认是否能够响应请求，结果如图 5-1 所示，显示合约地址说明调用成功，RBFT 块号当时为 3，说明已经出块了。

```
lll@ubuntu:~$ ./deploy.py
Deploy the smartContract, deploy.py will then trigger tps.py to START counting.
Logging into file logs/deploy.py.log.

{filedate 1556834478} new contract address: 0x77EEHRW7837997DD79G8A890DF78ASS97
8C5SDX5

blocknumber_start_here = 3
starting timer, at block 3 which has 1 transactions; at epochtime 1550534479.19
95363
```

图 5-1 RBFT 响应请求

5.2.1 TPS 实验

本实验的实验目标是得出 RBFT 在最佳情况下的 TPS 平均值。

类比 3.3 节中的实验，首先部署 7 个节点在共识算法为 RBFT 的 Quorum 联盟链中，分别用单线程、3 个线程、5 个线程、10 个线程、20 个线程、50 个线程与 100 个线程作为制造交易的 worker 同时发送交易，通过调用 web3 的 getBlockTransactionCount 接口得以统计新块包含的交易数，通过时间戳得到发送交易的总时间，由公式 3.2 即可得出每次实验的平均 TPS。在单线程发送交易时，RBFT 的平均 TPS 只有 23 左右，如图 5-2 所示。

```

block 1237 | new #TX 10 / 461 ms = 21.7 TPS_current | total: #TX 2775 / 120.9 s = 23.0 TPS_average
block 1244 | new #TX 20 / 459 ms = 43.6 TPS_current | total: #TX 2795 / 121.3 s = 23.0 TPS_average
block 1248 | new #TX 13 / 335 ms = 38.8 TPS_current | total: #TX 2808 / 121.8 s = 23.0 TPS_average
block 1250 | new #TX 15 / 648 ms = 23.2 TPS_current | total: #TX 2823 / 122.4 s = 23.1 TPS_average
block 1255 | new #TX 9 / 295 ms = 30.5 TPS_current | total: #TX 2832 / 122.8 s = 23.1 TPS_average
block 1261 | new #TX 14 / 559 ms = 25.1 TPS_current | total: #TX 2846 / 123.1 s = 23.1 TPS_average
block 1267 | new #TX 6 / 211 ms = 28.5 TPS_current | total: #TX 2852 / 123.5 s = 23.1 TPS_average
block 1270 | new #TX 15 / 278 ms = 54.0 TPS_current | total: #TX 2867 / 124.0 s = 23.1 TPS_average
block 1273 | new #TX 2 / 159 ms = 12.6 TPS_current | total: #TX 2869 / 124.3 s = 23.1 TPS_average
block 1275 | new #TX 4 / 420 ms = 9.5 TPS_current | total: #TX 2873 / 124.9 s = 23.0 TPS_average
block 1277 | new #TX 19 / 927 ms = 20.5 TPS_current | total: #TX 2892 / 125.4 s = 23.1 TPS_average
block 1289 | new #TX 11 / 341 ms = 32.3 TPS_current | total: #TX 2903 / 125.8 s = 23.1 TPS_average

```

图 5-2 RBFT 单线程 TPS

实验结果如图 5-3 所示。

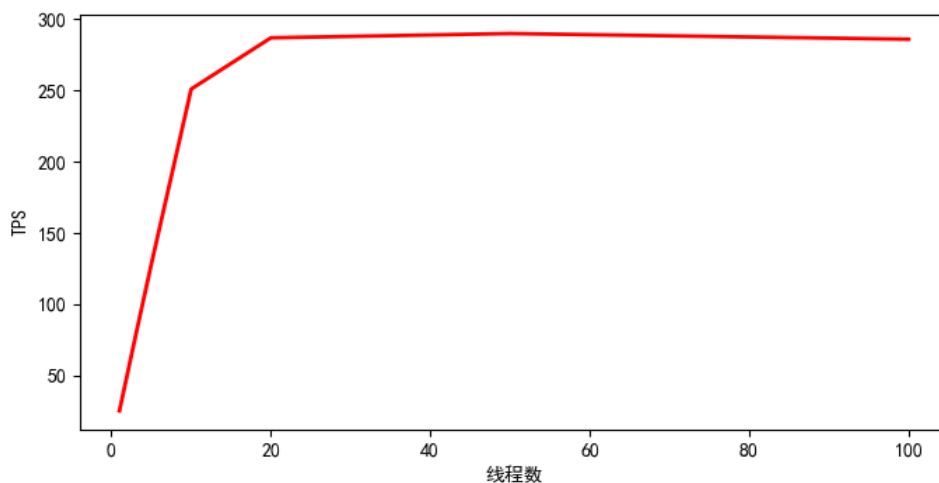


图 5-3 RBFT 线程数与 TPS 关系

由图可知在大于 20 个线程同时发送交易时 RBFT 的平均 TPS 基本不再变化，故保守取 30 个线程同时发送交易，这样也能保证与 3.3 节一致。由于 RBFT 的实现中对区块进行了一定的修改，所以仍需对区块大小进行实验，因为前文提到区块大小会影响区块链的性能。以太坊的区块大小由第二章里的区块字段 gasLimit 进行控制，故在 30 个线程发送交易的条件下修改 gasLimit 对 RBFT 进一步实验。

Quorum 的默认值为 4.7×10^6 ，为使实验结果具有可比性，仍取 10^5 , 3×10^5 , 10^6 , 2×10^6 , 4.7×10^6 , 10^7 , 2×10^7 , 3×10^7 分别进行实验。TPS 与 gasLimit 的关系如图 5-4。

由图可知在 RBFT 算法实现后, gasLimit 对 TPS 还是有较大影响, 故 gasLimit 依旧选择 2×10^7 确保出块正常。

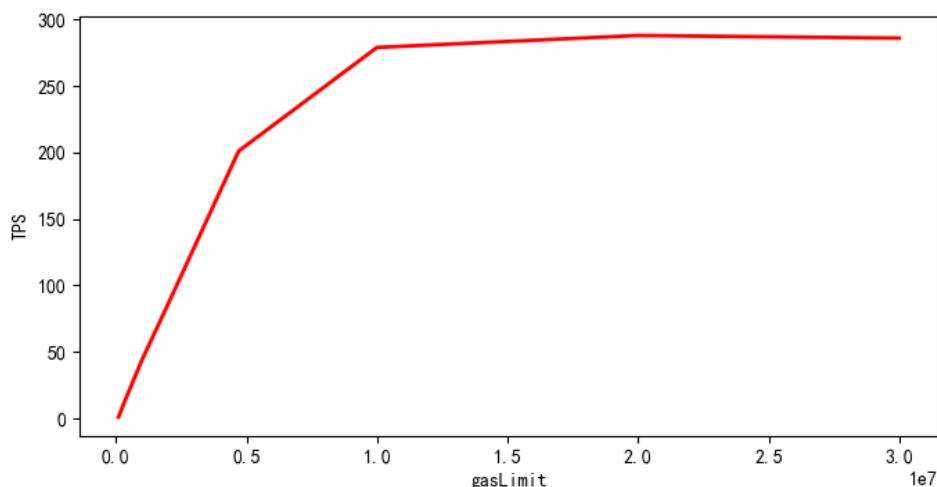


图 5-4 RBFT 中 TPS 与 gasLimit 关系

确认最佳实验环境后, RBFT 某次实验稳定后的 TPS 表现如图 5-5 所示。

```

block 141 | new #TX 288 / 1000 ms = 288.0 TPS_current | total: #TX 14803 / 51.4
s = 282.4 TPS_average
block 142 | new #TX 285 / 1000 ms = 285.0 TPS_current | total: #TX 15088 / 52.6
s = 286.8 TPS_average
block 143 | new #TX 283 / 1000 ms = 283.0 TPS_current | total: #TX 15371 / 53.5
s = 287.3 TPS_average
block 144 | new #TX 284 / 1000 ms = 284.0 TPS_current | total: #TX 15655 / 54.4
s = 287.2 TPS_average
block 145 | new #TX 290 / 1000 ms = 290.0 TPS_current | total: #TX 15945 / 55.6
s = 286.8 TPS_average
block 146 | new #TX 287 / 1000 ms = 287.0 TPS_current | total: #TX 16232 / 56.5
s = 287.3 TPS_average
block 147 | new #TX 291 / 1000 ms = 291.0 TPS_current | total: #TX 16523 / 57.7
s = 286.4 TPS_average
block 148 | new #TX 286 / 1000 ms = 286.0 TPS_current | total: #TX 16809 / 58.6
s = 286.8 TPS_average
block 149 | new #TX 285 / 1000 ms = 285.0 TPS_current | total: #TX 17094 / 59.4
s = 287.8 TPS_average
block 150 | new #TX 283 / 1000 ms = 283.0 TPS_current | total: #TX 17377 / 60.7
s = 286.3 TPS_average
block 151 | new #TX 287 / 1000 ms = 287.0 TPS_current | total: #TX 17664 / 61.6
s = 286.8 TPS_average

```

图 5-5 RBFT 的 TPS 表现

为了不失一般性还需对 RBFT 进行多次实验。取 30 个线程作为 30 个 worker 同时发送交易, gasLimit 取 2×10^7 , 进行 12 次性能实验。实验结果如图 5-6 所示, RBFT 平均 TPS 经过计算为 285.7。

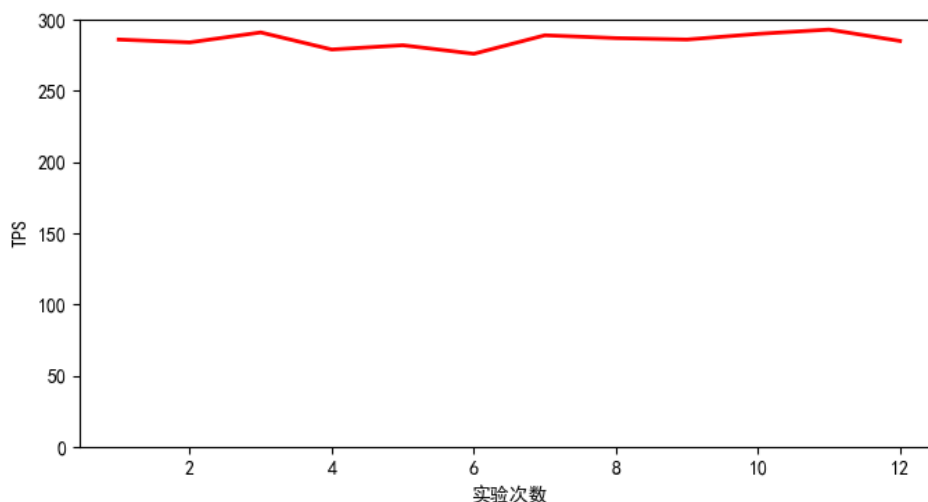


图 5-6 RBFT 七节点 TPS

RBFT 在扩展性上提升很大，所以依次增加至 8 台服务器进行实验，每次增加 7 个节点后重复 12 次实验得到平均 TPS 结果。性能如图 5-7 所示，RBFT 并未在节点增多时 TPS 有明显下降，始终维持在正常水平。

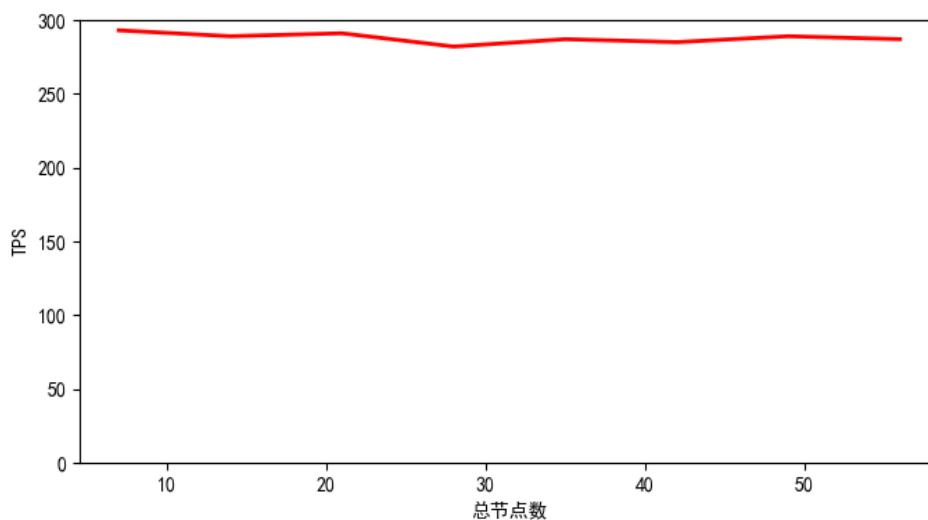


图 5-7 RBFT 节点数量与 TPS 关系

5.2.2 共识确认时间实验

本实验得出 RBFT 运行时的共识确认时间，主要目的在于确认共识确认时间在节点数大幅增加后是否会明显变长。

根据公式 3.3，在共识算法满负载运行情况下，通过记录某一批交易的发送时刻与这些交易被打包进区块的时刻可以求得共识确认时间，为得到正确的实验结果需要保证产生的新区块的大小不是性能瓶颈，所以 gasLimit 取 2×10^7 。提案者一旦确认它的提案区块已经上链，便利用交易在发送时加上的时间戳计算共识确认时间。

实验采用 20 个线程同时发送交易，能使 RBFT 处于满负荷状态。用于实验

的交易在新区块上链时发出,其余交易使共识算法始终保持满负载运行。第一次实验在 7 节点的单机进行,重复 5 次后计算得到共识确认时间 T_{cc} 的平均值。随后每次实验增加一台服务器直至 8 台,得到节点数与共识确认时间的关系。实验结果如图 5-8 所示。结果表明,在节点增加后,RBFT 的共识确认时间仍然足够小,保持在 0.7s 以内。这是因为 RBFT 在引入 VRF 后具有高扩展性,始终使得共识节点规模不会超过导致性能下降的阈值,共识算法能够一直高效地运行。

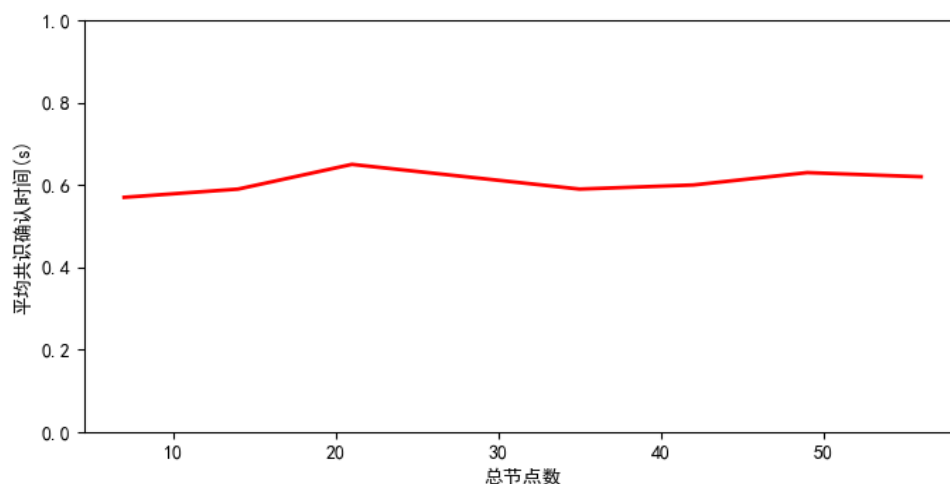


图 5-8 RBFT 共识确认时间

5.3 拜占庭容错实验

假设各节点的权益一致,此时 VRF 中签概率相同。使用 8 台服务器,每台部署 7 个节点,组成共 56 个节点的区块链网络。

为 56 个节点赋相同的权益值,并逐渐增加拜占庭节点的数量直至 RBFT 无法共识。进行 10 次实验,每次实验后记录导致区块链无法共识的拜占庭节点最小数量,实验结果如图 5-9。

设总节点数为 $3f+1$,理论上 RBFT 的拜占庭容错能力与 PBFT 是相同的,拜占庭节点数量小于等于 f 时共识算法能够正常运行。然而由于 VRF 使得每次共识最终参与的共识节点是随机分布的,出现了小于 18 个拜占庭节点时也偶尔会无法达成共识以及大于 18 个拜占庭节点却能维持数轮正常共识的情况,这种波动对于拜占庭容错能力来说影响不大,因为攻击者无法预测共识节点抽签结果,所以无法预测到哪些拜占庭节点将被抽中,意味着攻击者的攻击成本很高,需要控制非常多的高权重节点。总体来说,RBFT 的拜占庭容错能力与 PBFT 基本一致,能容忍小于总数 $1/3$ 的拜占庭节点,在联盟链中已经足够。

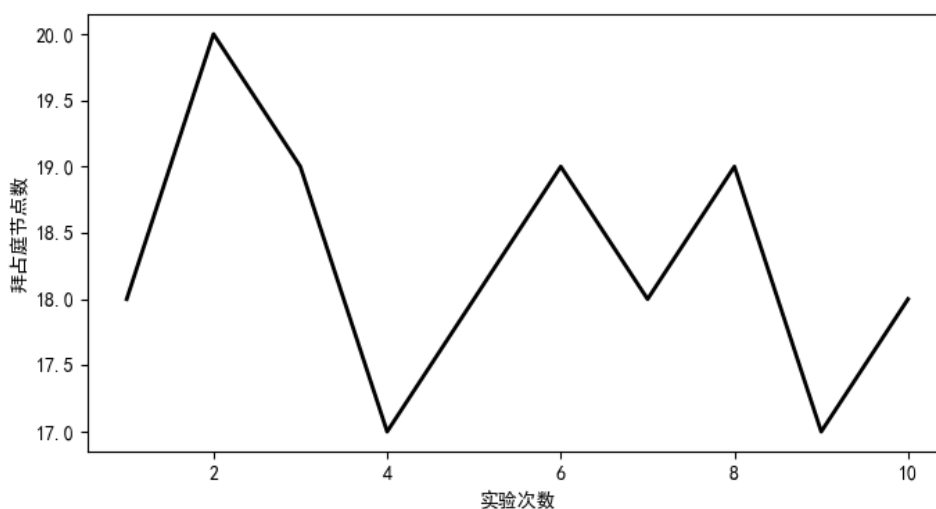


图 5-9 RBFT 拜占庭容错实验

5.4 实验结果分析

RBFT 与 PBFT、Raft、PoW 的实验结果对比如表 5-1。

表 5-1 Raft 与经典算法对比

共识算法 指标	RBFT	PBFT	Raft	PoW
TPS	287	211	188	7
共识确认时间	0.6 秒	Fabric: 1 秒 FISCO-BCOS: 1 秒	FISCO-BCOS: 1 秒	比特币: 60 分钟 以太坊: 1 分钟
共识节点数量	全部节点	小于 30	小于 30	全部节点
拜占庭容错能力	小于 1/3 总节点数	小于 1/3 总节点数	无	小于 1/2 总节点数
资源消耗	低	低	低	高

经过一系列改进，RBFT 的性能相对 PBFT 与 Raft 有较大提升，重要的是，RBFT 在共识节点数量增加时，并不会会有明显的性能下降，相比 PBFT 与 Raft 而言还提升了扩展性。RBFT 与 PoW 相比无论是 TPS 还是共识确认时间都有巨大优势，且基本没有资源消耗，在拜占庭容错能力上虽然相对更弱，但在联盟链或一般的公有链场景下已经足够。

综上，RBFT 在性能以及扩展性上有出色表现，拜占庭容错能力与 PBFT 一致，共识算法的公平性也有理论支撑，是适用于联盟链与公有链的优秀共识算法。

第六章 总结与展望

6.1 总结

共识算法是区块链的核心，而当前主流的共识算法都有明显问题。例如比特币所使用的 PoW 虽然提供了拜占庭容错的能力，但在共识过程中强大算力造成了资源浪费，且其交易确认时间长达 10 分钟。PBFT 适合应用于联盟链，但其违背去中心化的 C/S 架构以及不支持共识节点的动态加入与退出都是有待解决的问题。Raft 是崩溃容错共识算法的代表，然而其实现复杂且没有拜占庭容错能力，难以在公有链与联盟链中应用。

为尽可能解决这些问题，本文提出一种基于 VRF 的拜占庭容错共识算法 RBFT。主要工作如下：

1. 对三个经典共识算法进行了详尽的实验，通过分析实验结果总结出了改进方向；
2. 对 PBFT 进行了多项改进，主要有节点增减动态化、替换为 P2P 网络模型，还有一些细节改进以加速共识流程；
3. 算法设计中引入 VRF 与 PoS，VRF 带来了高扩展性以及公平性，PoS 提升了 RBFT 的公平性与安全性；
4. 将 RBFT 实现在类以太坊的 Quorum 中，得以直接应用于企业联盟链。

经过实验评估得到 RBFT 的实验结果，相比 PBFT、Raft、PoW 等算法都有明显的优势，尤其适合用于联盟链场景。

本文的创新体现如下：

- 1) 将 VRF 实现于共识算法中，合理利用 VRF 进行抽签以及验证，使得共识节点的产生是公平可信的，同时提升了共识算法的扩展性，在节点数量大时性能优势明显。
- 2) PoS 的权益与 VRF 相结合，防范了女巫攻击，能为商业应用提供公平的激励机制。
- 3) 改进了 PBFT 的固有缺陷，采用 Gossip 网络进行节点连接，优化了 PBFT 的共识流程，最终提升了 RBFT 的性能。

6.2 展望

本文提出并实现了 RBFT，综合性能、扩展性与安全性三点，RBFT 的实际表现比大部分共识算法在联盟链场景下更有优势，尤其是在节点数较多的情况下。不过由于区块链本身架构庞大，交叉领域众多，例如在有代币的链中，有足够的经济学知识才能保证整个系统的自治；又如密文的消息通讯过程需要有密码学背景才能确保其安全。因此，本文难以对共识算法的改进做到面面俱到，故对后续研究有如下展望：

- 1) 从博弈论的角度完善共识算法。区块链最出色的应用比特币标榜去中心化，而实际情况是比特币的主链产生的代币基本都被几个矿池包揽，因为运算力低下的节点基本不可能完成计算得到代币奖励，所以少数矿池垄断了比特币，去中心化的特征基本消失。造成这种现象的根本原因是比特币的激励机制不够合理，缺乏博弈论的考量。其它经典的应用如以太坊、EOS 等也都有类似的问题存在。所以从博弈论角度对共识算法进行更好地设计是很有必要的，比如加入完备的投票决策机制。
- 2) VRF 抽签过程的公平性。VRF 产生的随机哈希值直接决定了共识算法某一轮的参与者，所以必须确保随机函数的值域是离散、均匀分布的，另外在极端情况（如抽签结果只有一个重复中签的节点），共识算法需要有相应的处理机制保证共识的安全与正确。在 VRF 的提案节点被攻击时，也需要临时的主节点选举策略。这些由于难以复现，本文并未深入研究。
- 3) 共识算法的安全性。共识算法涉及到许多签名与消息通讯，需要用到加密库以及序列化库，这些技术的选择都会影响共识算法的安全性，涉及到运行速度与安全性的取舍。另外隐私保护也十分重要，需要有一套机制保证参与者的信息不会被攻击者获取。

致谢

逝者如斯夫，不舍昼夜。转眼间，我的研究生生涯已经接近尾声，回首过去，感触颇多。这是一段色彩斑斓的时光，既有科研带来的巨大压力，找工作途中的坎坷，也有同学间的欢声笑语，以及知识累积带来的充实感。经过这番历练与总结，我遇到了一个更优秀的自己，也认识到，自己的成长与那些陪我成长的人密不可分，值此论文付梓之际，谨向曾经给予我关心和帮助的师长、家人和朋友，致以最崇高的感谢。

首先感谢我的导师李必信教授。李教授平易近人，无论是讨论学术问题还是生活问题，总是谦和有礼，让人如沐春风；他拥有广博的学术知识，能够一针见血地看到问题的症结所在，但在讲述之时却又不咄咄逼人，让人觉得成为学术大家莫过若此，是我等后辈学习的楷模。我能够取得今天的研究成果，离不开李教授的悉心指导和其科研精神的感染。

其次，我还要感谢叶鹏老师，作为我的企业指导老师，他可谓呕心沥血，殚精力竭。不管平日工作多忙，当我遇到问题并向他请教时，他都会及时与我讨论问题，给出有价值的建议。他不仅以极强的工作能力帮助了我，更通过独特的解决问题之道深深感染了我，相信从他身上所学到的这些东西将使我终身受用。

感谢 418 实验室的同学们以及我的室友，创造了良好的学习氛围，互相督促，共同进步，还做到了劳逸结合，在学习感到疲惫时，大家一起进行乒乓球、篮球等运动来解压，这让我的科研生活变得丰富多彩，激发出了更多的灵感。知音难求，在学生时代的最后一段旅程中遇到大家是一份幸运，这将是我人生中难忘的美好时光。

最后，我要深深感谢我的家人和朋友。感谢亲爱的父亲、母亲，这么多年来始终如一无私地关心我、支持我、鼓励我，让我在砥砺前行路上沐浴着阳光，充满着希望。也感谢其他支持我的亲戚和朋友，不论远在他乡，还是就在身旁，感谢给我的关心和挂念。感谢我的女友，一个彩虹般绚烂的人，从始至终关心着我，支持着我。

参考文献

- [1] 袁勇, 王飞跃. 区块链技术发展现状与展望[J]. 自动化学报, 2016, 42(4).
- [2] Nakamoto S. Bitcoin: A peer-to-peer electronic cash system[EB]. 2008. <https://www.bitcoin.org/bitcoin.pdf>.
- [3] Lamport L, Shostak R, Pease M. The Byzantine generals problem[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 1982, 4(3): 382-401.
- [4] Lamport L. The part-time parliament[J]. ACM Transactions on Computer Systems (TOCS), 1998, 16(2): 133-169.
- [5] Lamport L. Paxos made simple[J]. ACM Sigact News, 2001, 32(4): 18-25.
- [6] Gafni E, Lamport L. Disk paxos[C]//International Symposium on Distributed Computing. Springer, Berlin, Heidelberg, 2000: 330-344.
- [7] Lamport L, Massa M. Cheap paxos[C]//Dependable Systems and Networks, 2004 International Conference on. IEEE, 2004: 307-314.
- [8] Lamport L. Fast paxos[J]. Distributed Computing, 2006, 19(2): 79-103.
- [9] Burrows M. The Chubby lock service for loosely-coupled distributed systems[C]//Proceedings of the 7th symposium on Operating systems design and implementation. USENIX Association, 2006: 335-350.
- [10] Chandra T D, Griesemer R, Redstone J. Paxos made live: an engineering perspective[C]//Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing. ACM, 2007: 398-407.
- [11] Ongaro D, Ousterhout J K. In search of an understandable consensus algorithm[C]//USENIX Annual Technical Conference. 2014: 305-319.
- [12] Oki B M, Liskov B H. Viewstamped replication: A new primary copy method to support highly-available distributed systems[C]//Proceedings of the seventh annual ACM Symposium on Principles of distributed computing. ACM, 1988: 8-17.
- [13] 袁勇, 倪晓春, 曾帅, 王飞跃. 区块链共识算法的发展现状与展望[J]. 自动化学报, 2018, 44(11): 2011-2022.
- [14] Bracha G, Toueg S. Asynchronous consensus and broadcast protocols[J]. Journal of the ACM (JACM), 1985, 32(4): 824-840.
- [15] Canetti R, Rabin T. Fast asynchronous Byzantine agreement with optimal resilience[C]//Proceedings of the twenty-fifth annual ACM symposium on Theory of computing. ACM, 1993: 42-51.
- [16] Malkhi D, Reiter M. Unreliable intrusion detection in distributed computations[C]//csfw. IEEE, 1997: 116.
- [17] Reiter M K. Secure agreement protocols: Reliable and atomic group multicast in Rampart[C]//Proceedings of the 2nd ACM Conference on Computer and Communications Security. ACM, 1994: 68-80.
- [18] Castro M, Liskov B. Practical Byzantine fault tolerance[C]//OSDI. 1999, 99: 173-

- 186.
- [19]Copeland C, Zhong H. Tangaroa: a byzantine fault tolerant raft[J]. 2014,3.
- [20]Gilad Y, Hemo R, Micali S, et al. Algorand: Scaling byzantine agreements for cryptocurrencies[C]//Proceedings of the 26th Symposium on Operating Systems Principles. ACM, 2017: 51-68.
- [21]Dwork C, Naor M. Pricing via processing or combatting junk mail[C]//Annual International Cryptology Conference. Springer, Berlin, Heidelberg, 1992: 139-147.
- [22]Back A. Hashcash-a denial of service counter-measure[J]. 2002,2.
- [23]Jakobsson M, Juels A. Proofs of work and bread pudding protocols[M]//Secure Information Networks. Springer, Boston, MA, 1999: 258-272.
- [24]Eyal I, Gencer A E, Sirer E G, et al. Bitcoin-NG: A Scalable Blockchain Protocol[C]//NSDI. 2016: 45-59.
- [25]Kogias E K, Jovanovic P, Gailly N, et al. Enhancing bitcoin security and performance with strong consistency via collective signing[C]//25th USENIX Security Symposium (USENIX Security 16). 2016: 279-296.
- [26]Luu L, Narayanan V, Zheng C, et al. A secure sharding protocol for open blockchains[C]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2016: 17-30.
- [27]Kokoris-Kogias E, Jovanovic P, Gasser L, et al. OmniLedger: A Secure, Scale-Out, Decentralized Ledger[J]. IACR Cryptology ePrint Archive, 2017, 2017: 406.
- [28]King S, Nadal S. Ppcoin: Peer-to-peer crypto-currency with proof-of-stake[J]. self-published paper, August, 2012, 19.
- [29]Miller A, Xia Y, Croman K, et al. The honey badger of BFT protocols[C]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2016: 31-42.
- [30]David B M, Gazi P, Kiayias A, et al. Ouroboros Praos: An adaptively-secure, semi-synchronous proof-of-stake protocol[J]. IACR Cryptology ePrint Archive, 2017, 2017: 573.
- [31]Larimer D. Delegated proof-of-stake (dpos)[J]. Bitshare whitepaper, 2014,3.
- [32]张伟.ECDSA 算法实现及其安全性分析[J].信息与电子工程,2003(02):7-12.
- [33]Koblitz N. Elliptic Curve Cryptosystems[J]. Mathematics of Computation, 1987, 48(177):203-209.
- [34]Miller V S. Use of Elliptic Curves in Cryptography[J]. Advances in Cryptology-CRYPTO'85, 1986, 19(03):173--193.
- [35]Johnson D, Menezes A, Vanstone S. The Elliptic Curve Digital Signature Algorithm (ECDSA)[J]. International Journal of Information Security, 2001, 1(1):36-63.
- [36]邵奇峰,金澈清,张召,钱卫宁,周傲英.区块链技术:架构及进展[J].计算机学报,2018,41(05):969-988.
- [37]范捷,易乐天,舒继武.拜占庭系统技术研究综述[J].软件学报,2013,24(06):1346-

- 1360.
- [38]Douceur J R. The sybil attack[C]//International workshop on peer-to-peer systems. Springer, Berlin, Heidelberg, 2002: 251-260.
- [39]Micali S, Vadhan S, Rabin M. Verifiable Random Functions[J]. Foundations of Computer Science Annual Symposium on, 1999:120-130.

